

claude-windows / c7301ee3-995d-4127-8dbb-e39cccaeeb08

Metadata

```
- Source: `claude-windows`  
- Kind: `claude`  
- Source file: `C:\Users\avidu\.claude\projects\C--Users-avidu-Projects-badminton-highlight-indexer\c7301ee3-995d-4127-8dbb-e39cccaeeb08.jsonl`  
- SHA-256: `0fef2ae136880c6a89ac42356e611bcf0133849d57e6a71abfab014bc0b73ece`  
- Source modified: `2026-06-22T09:13:33+00:00`  
- Imported at: `2026-07-05T16:48:19+00:00`  
- project: `C--Users-avidu-Projects-badminton-highlight-indexer`  
- session_id: `c7301ee3-995d-4127-8dbb-e39cccaeeb08`
```

Transcript

1. user (2026-06-18T01:07:07.482Z)

In the badminton-highlight-indexer repo, two golden-feature loaders have the SAME latent bug just fixed in `backend/eval/ablation.py::load\_videos` (commit on branch `fix/ablation-litmus-mixed-schema`): they read `c["shuttle"]`/`c["person"]` unconditionally on every candidate, so they crash with `KeyError: 'shuttle'` on a MIXED golden corpus where some `output/\*\_golden\_features.json` files carry the leaner velocity-windowing/distill schema (keys: active_frame_count, chunk_index, end, mean_velocity, peak_velocity, start, track_id_count ? no shuttle/person/audio cue dicts).

Affected:

```
- `backend/eval/fusion_compare.py::load_videos` (~line 39-48) ? `python -m  
backend.eval.fusion_compare --features-dir output` would KeyError on the n=11 corpus.  
- `backend/eval/fusion_audio.py::load_videos_with_audio` (~line 56-69) ? same, plus it reads  
`c["audio"]` unconditionally too.
```

Unlike the ablation litmus, these are NOT caught by any test today because `tests/test\_fusion\_compare.py` and `tests/test\_audio\_features.py` only exercise synthetic tmp_path corpora, never the real `output/`. So this is latent, not currently red ? but it bites the moment someone runs those CLIs on the real mixed corpus.

Fix: mirror the ablation fix ? reuse/duplicate the `\_is\_fusion\_schema(candidates)` predicate (full-fusion iff every candidate has `shuttle`+`person`; audio optional) and skip leaner windowing-only files with an info log instead of crashing. Consider whether `\_is\_fusion\_schema` should move to a shared spot (e.g. it currently lives in `backend/eval/ablation.py`) vs. being duplicated ? keep it simple, DRY only if it doesn't create an awkward import. Add a regression test per loader (mixed corpus loads only the fusion files). Keep it offline/mockd.

Verify: full suite (`python run\_all\_tests.py`), `ruff check .`, `mypy backend training` all clean; coverage stays ?70 (currently ~80.7%). Branch from `origin/master` in an isolated worktree, re-verify HEAD before commit, stage explicit paths (shared checkout ? sibling sessions share it). ONE PR per work item; do not stack on the ablation fix.

2. assistant (2026-06-18T01:07:11.257Z)

I'll start by understanding the existing fix in `ablation.py` and the two affected loaders, then set up an isolated worktree off `origin/master`.

3. user (2026-06-18T01:07:15.896Z)

```
be39a70 docs: rally-detection quality report ? technical-paper checkpoint (2026-06-17) (#206)  
---current branch---  
docs/golden-regression-fixtures-plan  
---status---  
?? artifacts/  
?? docs/KHELSUTRA_PRODUCT_OVERVIEW.md
```

4. assistant (2026-06-18T01:07:21.165Z)

Let me examine the existing fix in `ablation.py` and the two affected files.

5. user (2026-06-18T01:07:22.384Z)

```
25:A signal can be a **feature column** (person/audio/shuttle columns the learned scorer
weights) or
35:   the golden corpus may lack (Gate-A vs held-shuttle FPs) is marked ``structural`` and is
NEVER
62:# Source: distill_local.FEATURE_NAMES (shuttle, includes net_crossings), fusion_features
63:# (person), audio_features (audio). The shuttle group's velocity block + duration are the
64:# learned-column shuttle signals; net_crossings is BOTH a column and the Gate-A gate knob.
68:   "in_court_ratio", "shuttle_player_colocation")
90:   - ``name`` ? table/row identity ("person.colocation" | "gate_a" | "audio").
91:   - ``group`` ? producer ("person" | "gate_a" | "audio" | "shuttle"). LOGO ablates a whole
group.
93:   pure gate signal like Gate-A whose column is shared with the shuttle group).
553:   ""The current 4-group inventory (LOSO granularity): shuttle (velocity block +
duration),
354:   gate_a (the structural net-crossings gate AND its column), person, audio.
356:   Gate-A is marked ``structural`` (the held-shuttle FP guard) so it is reported but never
360:   Signal("shuttle.duration", "shuttle", ("duration",)),
361:   Signal("shuttle.velocity", "shuttle", ("peak_velocity", "mean_velocity",
"active_ratio")),
363:   Signal("person", "person", PERSON_COLUMNS),
386:def load_videos(features_dir: str) -> List[VideoCandidates]:
387:   ""Load every ``*_golden_features.json`` into a ``VideoCandidates`` (intervals + shuttle
+
388:   person + audio columns + golden gts). Audio columns are optional (older JSONs predate
the
399:   features[fn] = [float(c["shuttle"][fn]) for c in cand]
401:   features[fn] = [float(c["person"][fn]) for c in cand]
411:   ""The learned full-set reference: all 13 columns (shuttle 5 + person 5 + audio 3)
weighted
553:   "corpus may lack (e.g. Gate-A vs held-shuttle false positives); reported honestly
but "
```

6. user (2026-06-18T01:07:28.991Z)

```
+ fix/ablation-litmus-mixed-schema
---
```

7. assistant (2026-06-18T01:07:35.883Z)

`\_is\_fusion\_schema` isn't on my current branch. Let me look at the fix branch to understand the pattern, and check whether it's merged to master.

8. user (2026-06-18T01:07:38.019Z)

```
=== is fix branch merged to origin/master? ===
NOT merged to master
```

```
=== diff of fix branch vs origin/master on ablation.py ===
diff --git a/backend/eval/ablation.py b/backend/eval/ablation.py
index fec3e7d..968eefd 100644
--- a/backend/eval/ablation.py
+++ b/backend/eval/ablation.py
@@ -383,16 +383,37 @@ def litmus_config() -> AblationConfig:
# ----- corpus I/O

+def _is_fusion_schema(candidates: Sequence[Dict[str, Any]]) -> bool:
+   ""True iff every candidate row carries the per-cue feature dicts the ablation scorer reads
```

```

+     (`shuttle` + `person`; `audio` stays optional ? older JSONs predate it).
+
+     The velocity-windowing / distill path persists a LEANER ``*_golden_features.json`` row
+     (`start`/`end` + velocity stats, no cue dicts) for clips that never went through fusion
+     feature extraction. ``load_videos`` SKIPS those rather than crash on ``c["shuttle"]`` ? so
+     a
+     mixed corpus (some fusion files, some windowing-only) loads the ablatable subset cleanly.
+     An empty candidate list is not ablatable either."""
+     return bool(candidates) and all("shuttle" in c and "person" in c for c in candidates)
+
+
+     def load_videos(features_dir: str) -> List[VideoCandidates]:
+         """Load every ``*_golden_features.json`` into a ``VideoCandidates`` (intervals + shuttle +
+         person + audio columns + golden gts). Audio columns are optional (older JSONs predate the
+         audio augment) ? a missing block defaults those columns to 0.0 so the loader never crashes,
+         mirroring ``experiment._feature_column``'s tolerance."""
+         """Load every FULL-FUSION-SCHEMA ``*_golden_features.json`` into a ``VideoCandidates``
+         (intervals + shuttle + person + audio columns + golden gts). Audio columns are optional
+         (older JSONs predate the audio augment) ? a missing block defaults those columns to 0.0 so
+         the loader never crashes, mirroring ``experiment._feature_column``'s tolerance.
+
+         Leaner velocity-windowing-only files (no ``shuttle``/``person`` cue dicts ? the schema the
+         R3/R4 corpus-growth path persists) are SKIPPED, not crashed on, so a mixed corpus loads the
+         ablatable subset cleanly (see ``_is_fusion_schema``)."""
+         videos: List[VideoCandidates] = []
+         for path in sorted(glob.glob(os.path.join(features_dir, "*_golden_features.json"))):
+             with open(path, encoding="utf-8") as f:
+                 d = json.load(f)
+                 cands = d["candidates"]
+             if not _is_fusion_schema(cands):
+                 logger.info("ablation.load_videos: skipping %s ? windowing-only schema "
+                             "(no shuttle/person cue dicts), not an ablatable fusion-feature file",
+                             os.path.basename(path))
+                 continue
+             intervals = [(float(c["start"]), float(c["end"])) for c in cands]
+             features: Dict[str, List[float]] = {}
+             for fn in SHUTTLE_COLUMNS:

```

9. assistant (2026-06-18T01:07:48.444Z)

The fix branch is **not** merged to master, so importing ``\_is\_fusion\_schema`` from ``ablation.py`` would stack on an unmerged branch ? I'll keep it self-contained. Let me read both affected files in full.

10. user (2026-06-18T01:07:48.849Z)

```

1     """Pure-CPU LOVO comparison: shuttle-only vs shuttle+person learned fusion on the golden
2     set.
3     The fusion-P3 measurement. Loads the per-video feature JSONs written by
4     `fusion_golden.py`
5     and runs the SHIPPED experiment harness (`experiment.compare`, honest leave-one-video-
6     out) ?
7     no new scoring/labeling math. The two variants differ ONLY by `feature_names` (the
8     harness's
9     designed seam): the person features are extra columns the learned LogisticRegression
10    weights,
11    so the held-out ?F1 is an honest answer to "does the person cue help?" ? not a hunch.
12
13    Run (no GPU):
14    python -m backend.eval.fusion_compare --features-dir output --record
15
16    """
17    from __future__ import annotations
18
19    import argparse

```

```

15     import glob
16     import json
17     import os
18     from typing import Any, Dict, List
19
20     from backend.eval import experiment
21     from backend.eval.eval_stats import paired_delta_ci
22     from backend.eval.fusion_golden import PERSON_FEATURE_NAMES, SHUTTLE_FEATURE_NAMES
23
24
25     def delta_stats(res: Dict[str, Any], metric: str = "f1") -> Dict[str, Any]:
26         """Paired (by video) B?A summary for one metric: mean delta + bootstrap CI + sign
test
27         (eval_stats, C1 ? "never a bare mean"). Aligns the two variants' per-video held-out
scores
28         by video name, so the headline ?F1 comes with how-wide + did-it-win-on-enough-
videos."""
29         by_a = {p["video"]: p[metric] for p in res["variant_a"]["per_video"]}
30         by_b = {p["video"]: p[metric] for p in res["variant_b"]["per_video"]}
31         vids = [p["video"] for p in res["variant_a"]["per_video"]]
32         return paired_delta_ci([by_a[v] for v in vids], [by_b[v] for v in vids])
33
34
35     def load_videos(features_dir: str) -> List[experiment.VideoCandidates]:
36         """Load every ``*_golden_features.json`` into a VideoCandidates (intervals + the 10
37         feature columns + golden gts). The harness derives labels from gts itself."""
38         videos: List[experiment.VideoCandidates] = []
39         for path in sorted(glob.glob(os.path.join(features_dir, "*_golden_features.json"))):
40             with open(path, encoding="utf-8") as f:
41                 d = json.load(f)
42                 cands = d["candidates"]
43                 intervals = [(float(c["start"]), float(c["end"])) for c in cands]
44                 features: Dict[str, List[float]] = {}
45                 for fn in SHUTTLE_FEATURE_NAMES:
46                     features[fn] = [float(c["shuttle"][fn]) for c in cands]
47                 for fn in PERSON_FEATURE_NAMES:
48                     features[fn] = [float(c["person"][fn]) for c in cands]
49                 gts = [(float(a), float(b)) for a, b in d["gts"]]
50                 videos.append(experiment.VideoCandidates(name=d["name"], intervals=intervals,
51                                                         features=features, gts=gts))
52         return videos
53
54
55     def compare(videos: List[experiment.VideoCandidates], iou: float = 0.5,
56                threshold: float = 0.5) -> Dict[str, Any]:
57         shuttle_only = experiment.Variant(name="shuttle_only",
58                                           feature_names=list(SHUTTLE_FEATURE_NAMES),
59                                           threshold=threshold)
60         fusion = experiment.Variant(name="shuttle_person_fusion",
61                                     feature_names=list(SHUTTLE_FEATURE_NAMES) +
62                                     list(PERSON_FEATURE_NAMES),
63                                     threshold=threshold)
64         return experiment.compare(videos, shuttle_only, fusion, iou=iou, honest=True)
65
66     def format_result(res: Dict[str, Any], iou: float) -> str:
67         a, b, d = res["variant_a"]["aggregate"], res["variant_b"]["aggregate"], res["delta"]
68         ds = delta_stats(res)
69         st = ds["sign_test"]
70         lines = [
71             f"=== Fusion P3 LOVO (n={res['num_videos']} golden videos, IoU>={iou}, honest)
===",
72             f"    shuttle-only : F1={a['f1']:.3f}  P={a['precision']:.3f}  R={a['recall']:.3f}
IoU={a['mean_iou']:.3f}",
73             f"    +person      : F1={b['f1']:.3f}  P={b['precision']:.3f}  R={b['recall']:.3f}

```

```

IoU={b['mean_iou']:.3f}",
73         f" ?F1 = {d['f1']:+.3f} ({res['delta_pct']['f1']:+.1f}%)
?P={d['precision']:+.3f} ?R={d['recall']:+.3f}",
74         f" paired ?F1 {ds['mean_delta']:+.3f} [95% CI {ds['ci_low']:+.3f},
{ds['ci_high']:+.3f}] "
75         f"| sign test {int(st['wins'])}W/{int(st['losses'])}L (p={st['p_value']:.3f}) "
76         f"| CI clears 0: {ds['ci_excludes_zero']}",
77         " per-video ?F1:",
78     ]
79     for pv in res["per_video_delta"]:
80         lines.append(f" {pv['video']:<26s} ?F1={pv['f1']:+.3f}")
81     return "\n".join(lines)
82
83
84     def main() -> None:
85         ap = argparse.ArgumentParser(description="LOVO fusion lift: shuttle-only vs
shuttle+person (golden set).")
86         ap.add_argument("--features-dir", default="output")
87         ap.add_argument("--iou", type=float, default=0.5)
88         ap.add_argument("--record", action="store_true", help="append the result to the
experiment leaderboard")
89         args = ap.parse_args()
90
91         videos = load_videos(args.features_dir)
92         if len(videos) < 2:
93             raise SystemExit(f"need >=2 *_golden_features.json in {args.features_dir} (found
{len(videos)})")
94         res = compare(videos, iou=args.iou)
95         print(format_result(res, args.iou))
96         if args.record:
97             row = experiment.record_experiment(res)
98             print(f"recorded -> eval_baselines/experiment_leaderboard.json
({row['timestamp']}")
99
100
101     if __name__ == "__main__":
102         main()
103

```

11. user (2026-06-18T01:07:51.140Z)

```

1     """Fusion P4 ? measure the INCREMENTAL audio-cue lift on top of the person cue (CPU-
only).
2
3     Reuses the person-feature JSONs from `fusion_golden.py` (no GPU re-detect): `augment`
runs
4     `detect_hits_in_video` once per video (ffmpeg + numpy, CPU) and adds the 3 audio
features per
5     window; `compare_increment` then runs the SHIPPED harness between a shuttle+person
variant
6     and a shuttle+person+audio variant ? so audio is measured as a marginal contribution ON
TOP
7     of person, not against raw shuttle. Audio enters only as columns the learned scorer
weights
8     (no special-casing, default-OFF); the golden-set ?F1 decides whether it earns its keep.
9
10    Run (no GPU):
11        python -m backend.eval.fusion_audio --manifest output/human_lovo_manifest.json
--features-dir output --augment --compare
12    """
13    from __future__ import annotations
14
15    import argparse
16    import glob
17    import json

```

```

18     import os
19     from typing import Any, Dict, List
20
21     from backend.eval import experiment
22     from backend.eval.fusion_compare import delta_stats
23     from backend.eval.fusion_golden import PERSON_FEATURE_NAMES, SHUTTLE_FEATURE_NAMES,
_derive_video_path
24     from backend.pipeline.detectors import audio_features as af
25     from backend.pipeline.audio.hit_detector import detect_hits_in_video
26
27
28     def augment(manifest_path: str, features_dir: str, k: float = 2.5) -> List[str]:
29         """Add the 3 audio features to each existing ``<name>_golden_features.json``
window."""
30         with open(manifest_path, encoding="utf-8") as f:
31             specs = json.load(f)
32             written: List[str] = []
33             for spec in specs:
34                 jp = os.path.join(features_dir, f"{spec['name']}_golden_features.json")
35                 if not os.path.isfile(jp):
36                     print(f"[fusion_audio] {spec['name']}: no feature JSON ? run fusion_golden
first; skipping")
37                     continue
38                 video = _derive_video_path(spec)
39                 print(f"[fusion_audio] {spec['name']}: detecting audio hits (k={k})...",
flush=True)
40                 hits = detect_hits_in_video(video, k=k)
41                 with open(jp, encoding="utf-8") as f:
42                     data = json.load(f)
43                     for c in data["candidates"]:
44                         c["audio"] = af.compute_audio_features(hits, c["start"], c["end"])
45                 with open(jp, "w", encoding="utf-8") as f:
46                     json.dump(data, f, indent=2)
47                 print(f"[fusion_audio] {spec['name']}: {len(hits)} hits over
{len(data['candidates'])} windows", flush=True)
48                 written.append(jp)
49             return written
50
51
52     def load_videos_with_audio(features_dir: str) -> List[experiment.VideoCandidates]:
53         """Load the feature JSONs into VideoCandidates including the audio columns (requires
`augment` to have run ? raises if a JSON lacks audio)."""
54         videos: List[experiment.VideoCandidates] = []
55         for path in sorted(glob.glob(os.path.join(features_dir, "*_golden_features.json"))):
56             with open(path, encoding="utf-8") as f:
57                 d = json.load(f)
58                 cands = d["candidates"]
59                 if cands and "audio" not in cands[0]:
60                     raise SystemExit(f"{path} has no audio features ? run ``--augment`` first")
61                 intervals = [(float(c["start"]), float(c["end"])) for c in cands]
62                 features: Dict[str, List[float]] = {}
63                 for fn in SHUTTLE_FEATURE_NAMES:
64                     features[fn] = [float(c["shuttle"][fn]) for c in cands]
65                 for fn in PERSON_FEATURE_NAMES:
66                     features[fn] = [float(c["person"][fn]) for c in cands]
67                 for fn in af.AUDIO_FEATURE_NAMES:
68                     features[fn] = [float(c["audio"][fn]) for c in cands]
69                 gts = [(float(a), float(b)) for a, b in d["gts"]]
70                 videos.append(experiment.VideoCandidates(name=d["name"], intervals=intervals,
features=features, gts=gts))
71             return videos
72
73
74
75
76     def compare_increment(videos: List[experiment.VideoCandidates], iou: float = 0.5,
threshold: float = 0.5) -> Dict[str, Any]:

```

```

78     """Incremental audio lift: shuttle+person vs shuttle+person+audio (honest
LOVO)."""
79     person = experiment.Variant(
80         name="shuttle_person",
81         feature_names=list(SHUTTLE_FEATURE_NAMES) + list(PERSON_FEATURE_NAMES),
threshold=threshold)
82     person_audio = experiment.Variant(
83         name="shuttle_person_audio",
84         feature_names=list(SHUTTLE_FEATURE_NAMES) + list(PERSON_FEATURE_NAMES) +
list(af.AUDIO_FEATURE_NAMES),
85         threshold=threshold)
86     return experiment.compare(videos, person, person_audio, iou=iou, honest=True)
87
88
89     def format_result(res: Dict[str, Any], iou: float) -> str:
90         a, b, d = res["variant_a"]["aggregate"], res["variant_b"]["aggregate"], res["delta"]
91         ds = delta_stats(res)
92         st = ds["sign_test"]
93         lines = [
94             f"=== Fusion P4 incremental AUDIO LOVO (n={res['num_videos']}, IoU>={iou},
honest) ===",
95             f"  shuttle+person          : F1={a['f1']:.3f}  P={a['precision']:.3f}
R={a['recall']:.3f}",
96             f"  shuttle+person+audio : F1={b['f1']:.3f}  P={b['precision']:.3f}
R={b['recall']:.3f}",
97             f"  incremental ?F1 = {d['f1']:.3f}  ({res['delta_pct']['f1']:.1f}%)",
98             f"  paired ?F1 {ds['mean_delta']:.3f}  [95% CI {ds['ci_low']:.3f},
{ds['ci_high']:.3f}]  "
99             f"| sign test {int(st['wins'])}W/{int(st['losses'])}L (p={st['p_value']:.3f})  "
100            f"| CI clears 0: {ds['ci_excludes_zero']}",
101        ]
102        for pv in res["per_video_delta"]:
103            lines.append(f"    {pv['video']:<26s} ?F1={pv['f1']:.3f}")
104        return "\n".join(lines)
105
106
107     def main() -> None:
108         ap = argparse.ArgumentParser(description="Fusion P4 ? incremental audio lift on the
golden set.")
109         ap.add_argument("--manifest", default="output/human_lovo_manifest.json")
110         ap.add_argument("--features-dir", default="output")
111         ap.add_argument("-k", type=float, default=2.5, help="hit-detector sensitivity")
112         ap.add_argument("--augment", action="store_true", help="add audio features to the
existing JSONs")
113         ap.add_argument("--compare", action="store_true", help="report the incremental audio
LOVO ?F1")
114         ap.add_argument("--iou", type=float, default=0.5)
115         ap.add_argument("--record", action="store_true")
116         args = ap.parse_args()
117
118         if args.augment:
119             augment(args.manifest, args.features_dir, args.k)
120         if args.compare:
121             videos = load_videos_with_audio(args.features_dir)
122             if len(videos) < 2:
123                 raise SystemExit(f"need >=2 augmented feature files in {args.features_dir}")
124             res = compare_increment(videos, iou=args.iou)
125             print(format_result(res, args.iou))
126             if args.record:
127                 row = experiment.record_experiment(res)
128                 print(f"recorded -> eval_baselines/experiment_leaderboard.json
({row['timestamp']}")
129
130
131     if __name__ == "__main__":

```

```
132         main()
133
```

12. assistant (2026-06-18T01:08:05.085Z)

Both files use ``from typing import Any, Dict, List``, and ``fusion_audio`` already imports ``delta_stats`` from ``fusion_compare`` ? so importing a shared predicate from ``fusion_compare`` is a natural, non-awkward dependency. Let me check the logger setup in ``ablation.py`` and the existing tests.

13. user (2026-06-18T01:08:06.986Z)

```
51:import logging
59:logger = logging.getLogger(__name__)
=== fusion_compare logger? ===
=== Sequence import in ablation ===
54:from typing import Any, Dict, List, Optional, Sequence, Tuple
```

14. user (2026-06-18T01:08:10.555Z)

```
1      """GPU-free tests for the fusion-P3 measurement (fusion_golden extractor +
fusion_compare LOVO).
2
3      The LOVO driver is exercised on a synthetic 6-video set where the SHUTTLE features are
4      non-separating (constant) but a PERSON feature cleanly separates rallies ? so the
harness must
5      report a positive fusion ?F1. The extractor is exercised with a stubbed detector (no
GPU).
6      """
7      import json
8      import os
9      from types import SimpleNamespace
10
11      import pytest
12
13      from backend.eval import fusion_compare
14      from backend.eval.fusion_compare import PERSON_FEATURE_NAMES, SHUTTLE_FEATURE_NAMES
15
16
17      def _write_video(d, name):
18          """A video whose shuttle features are identical for pos/neg (no signal) but whose
19          players_in_court separates rallies (4) from junk (0)."""
20          cands = []
21          for (s, e), pos in [((0.0, 10.0), 1), ((20.0, 30.0), 1), ((40.0, 41.0), 0), ((50.0,
51.0), 0)]:
22              shuttle = {fn: 1.0 for fn in SHUTTLE_FEATURE_NAMES}          # constant
-> non-separating
23              person = {fn: 0.0 for fn in PERSON_FEATURE_NAMES}
24              person["players_in_court"] = 4.0 if pos else 0.0              # the
separating signal
25              person["in_court_ratio"] = 1.0 if pos else 0.0
26              cands.append({"start": s, "end": e, "shuttle": shuttle, "person": person,
"label": pos})
27              data = {"name": name, "fps": 30.0, "frame_width": 1920.0,
28                      "candidates": cands, "gts": [[0.0, 10.0], [20.0, 30.0]]}
29              with open(os.path.join(d, f"{name}_golden_features.json"), "w", encoding="utf-8") as
f:
30                  json.dump(data, f)
31
32
33      def test_fusion_compare_reports_person_driven_lift(tmp_path):
34          d = str(tmp_path)
35          for k in range(6):
36              _write_video(d, f"vid{k}")
```



```

37     videos = fusion_compare.load_videos(d)
38     assert len(videos) == 6
39     res = fusion_compare.compare(videos, iou=0.5)
40     assert res["num_videos"] == 6
41     assert res["variant_b"]["honest_lovo"] is True # LOVO-honest learned
variant
42     # Fusion (with the separating person feature) must beat shuttle-only (no signal).
43     assert res["delta"]["f1"] > 0.0
44     assert res["variant_b"]["aggregate"]["f1"] > res["variant_a"]["aggregate"]["f1"]
45     # format_result must not crash and must mention both variants.
46     out = fusion_compare.format_result(res, 0.5)
47     assert "shuttle-only" in out and "+person" in out
48
49
50     def test_fusion_compare_delta_stats_and_ci(tmp_path):
51         """delta_stats yields a paired bootstrap CI + sign test; format_result reports
them."""
52         d = str(tmp_path)
53         for k in range(6):
54             _write_video(d, f"vid{k}")
55         res = fusion_compare.compare(fusion_compare.load_videos(d), iou=0.5)
56         ds = fusion_compare.delta_stats(res)
57         assert set(ds) >= {"mean_delta", "ci_low", "ci_high", "sign_test",
"ci_excludes_zero"}
58         assert ds["ci_low"] - 1e-9 <= ds["mean_delta"] <= ds["ci_high"] + 1e-9 # CI
brackets mean (±fp)
59         assert ds["sign_test"]["wins"] == 6 # fusion wins on every video (person
feature separates)
60         assert ds["ci_excludes_zero"] is True
61         out = fusion_compare.format_result(res, 0.5)
62         assert "CI" in out and "sign test" in out
63
64
65     def test_fusion_compare_needs_two_videos(tmp_path):
66         _write_video(str(tmp_path), "only")
67         videos = fusion_compare.load_videos(str(tmp_path))
68         assert len(videos) == 1 # compare() / experiment.compare needs >=2 for honest LOVO
69
70
71     # ----- extractor assembly
72
73     def test_extract_video_assembles_shuttle_and_person(monkeypatch):
74         from backend.eval import fusion_golden
75         from backend.pipeline.detectors.tracknet_runner import TrackNetRunner
76
77         intervals = [(0.0, 10.0), (40.0, 41.0)]
78         x_shuttle = [[10.0, 0.3, 0.2, 0.5, 4.0], [1.0, 0.1, 0.05, 0.2, 0.0]]
79         points = [SimpleNamespace(frame=i, visible=True, x=500.0, y=(100.0 if i % 2 else
900.0))
80                     for i in range(10)]
81
82         monkeypatch.setattr(fusion_golden.distill_local, "build_candidates",
83                             lambda traj, fps, fw: (intervals, x_shuttle))
84         monkeypatch.setattr(TrackNetRunner, "parse_trajectory_csv", staticmethod(lambda
traj: points))
85         monkeypatch.setattr(fusion_golden, "load_golden_gts", lambda labels: [(0.0, 10.0)])
86
87         class _StubDetector:
88             def detections_over_windows(self, video, windows, frame_skip, progress_cb=None):
89                 return [{"frames": {sf: [(1, (10.0, 10.0, 30.0, 40.0))]},
90                             "fps": 30.0, "frame_width": 1920.0, "frame_height": 1080.0}
91                         for (sf, ef) in windows]
92
93         spec = {"name": "t", "trajectory": "bar.csv", "fps": 30.0, "frame_width": 1920.0,
94                 "labels": "foo.rallies.csv"}

```

```

95     data = fusion_golden.extract_video(spec, detector=_StubDetector(), frame_skip=3,
iou=0.5)
96
97     assert data["name"] == "t"
98     assert data["gts"] == [[0.0, 10.0]]
99     assert len(data["candidates"]) == 2
100    c0 = data["candidates"][0]
101    assert set(c0["shuttle"]) == set(SHUTTLE_FEATURE_NAMES)
102    assert set(c0["person"]) == set(PERSON_FEATURE_NAMES)
103    assert c0["label"] == 1 # (0,10) matches the gt -> positive
104    assert data["candidates"][1]["label"] == 0 # (40,41) -> negative
105    assert all(isinstance(v, float) for v in c0["person"].values())
106
107
108    def test_fusion_golden_run_is_resumable(tmp_path, monkeypatch):
109        """run() skips a video whose JSON already exists (resume a long run); --fresh redoes
all."""
110        from backend.eval import fusion_golden
111        out_dir = str(tmp_path / "out")
112        os.makedirs(out_dir)
113        manifest = [
114            {"name": "alpha", "trajectory": "a.csv", "fps": 30.0, "frame_width": 1920.0,
"labels": "a.rallies.csv"},
115            {"name": "beta", "trajectory": "b.csv", "fps": 30.0, "frame_width": 1920.0,
"labels": "b.rallies.csv"},
116        ]
117        manifest_path = str(tmp_path / "m.json")
118        with open(manifest_path, "w", encoding="utf-8") as f:
119            json.dump(manifest, f)
120        # a prior run already produced alpha's JSON
121        with open(os.path.join(out_dir, "alpha_golden_features.json"), "w",
encoding="utf-8") as f:
122            json.dump({"name": "alpha", "fps": 30.0, "frame_width": 1920.0, "candidates":
[], "gts": []}, f)
123
124        extracted = []
125
126        def _stub(spec, **kw):
127            extracted.append(spec["name"])
128            return {"name": spec["name"], "fps": 30.0, "frame_width": 1920.0, "candidates":
[], "gts": []}
129
130        monkeypatch.setattr(fusion_golden, "extract_video", _stub)
131
132        fusion_golden.run(manifest_path, out_dir, fresh=False)
133        assert extracted == ["beta"] # alpha skipped (already extracted)
134
135        extracted.clear()
136        fusion_golden.run(manifest_path, out_dir, fresh=True)
137        assert extracted == ["alpha", "beta"] # --fresh redoes both
138
139
140    def test_fusion_golden_smallest_first(tmp_path, monkeypatch):
141        """--smallest-first orders by trajectory size; default keeps manifest order."""
142        from backend.eval import fusion_golden
143        out_dir = str(tmp_path / "out")
144        os.makedirs(out_dir)
145        (tmp_path / "big.csv").write_text("x" * 2000)
146        (tmp_path / "small.csv").write_text("x" * 20)
147        manifest = [
148            {"name": "big", "trajectory": str(tmp_path / "big.csv"), "fps": 30.0,
"frame_width": 1920.0, "labels": "b.rallies.csv"},
149            {"name": "small", "trajectory": str(tmp_path / "small.csv"), "fps": 30.0,
"frame_width": 1920.0, "labels": "s.rallies.csv"},
150        ]

```

```

151     mp = str(tmp_path / "m.json")
152     with open(mp, "w", encoding="utf-8") as f:
153         json.dump(manifest, f)
154
155     order = []
156
157     def _stub(spec, **kw):
158         order.append(spec["name"])
159         return {"name": spec["name"], "fps": 30.0, "frame_width": 1920.0, "candidates":
160 [], "gts": []}
161
162     monkeypatch.setattr(fusion_golden, "extract_video", _stub)
163
164     fusion_golden.run(mp, out_dir, smallest_first=True)
165     assert order == ["small", "big"]          # smaller trajectory first
166
167     order.clear()
168     fusion_golden.run(mp, out_dir, smallest_first=False, fresh=True)
169     assert order == ["big", "small"]          # manifest order preserved
170
171     def test_derive_video_path():
172         from backend.eval.fusion_golden import _derive_video_path
173         assert _derive_video_path({"labels": "X/Adarsh and Avi.rallies.csv"}) == "X/Adarsh
174 and Avi.MP4"
175         assert _derive_video_path({"video_path": "a.mp4", "labels": "b.rallies.csv"}) ==
176 "a.mp4"
177         with pytest.raises(ValueError):
178             _derive_video_path({"name": "n", "labels": "weird.csv"})

```

15. user (2026-06-18T01:08:14.417Z)

```

1     """Tests for the P4 audio cue ? pure audio features + the incremental-lift driver (no
2 GPU/ffmpeg)."""
3     import json
4     import os
5     from types import SimpleNamespace
6
7     import pytest
8
9     from backend.eval import fusion_audio
10    from backend.pipeline.detectors import audio_features as af
11
12    def H(t, strength=1.0):
13        return SimpleNamespace(t=t, strength=strength)
14
15
16    def test_compute_audio_features_empty():
17        f = af.compute_audio_features([], 0.0, 10.0)
18        assert set(f) == set(af.AUDIO_FEATURE_NAMES)
19        assert all(v == 0.0 for v in f.values())
20
21
22    def test_compute_audio_features_counts_rate_strength():
23        hits = [H(1.0, 2.0), H(2.0, 4.0), H(5.0, 6.0), H(50.0, 1.0)]    # 3 in [0,10), 1
24    outside
25        f = af.compute_audio_features(hits, 0.0, 10.0)
26        assert f["audio_hit_count"] == 3.0
27        assert f["audio_hit_rate"] == pytest.approx(0.3)                # 3 hits / 10 s
28        assert f["audio_hit_strength_mean"] == pytest.approx((2 + 4 + 6) / 3)
29
30    def test_compute_audio_features_half_open_boundary():

```

```

31         hits = [H(10.0), H(20.0)]                                # window [10,20): 10
in, 20 out
32         f = af.compute_audio_features(hits, 10.0, 20.0)
33         assert f["audio_hit_count"] == 1.0
34
35
36         # ----- incremental-lift driver
(synthetic)
37
38         def _write_video(d, name):
39             """shuttle + person constant (no signal); audio_hit_count separates rallies (8) from
junk (0)."""
40             cand = []
41             for (s, e), pos in [((0.0, 10.0), 1), ((20.0, 30.0), 1), ((40.0, 41.0), 0), ((50.0,
51.0), 0)]:
42                 shuttle = {fn: 1.0 for fn in fusion_audio.SHUTTLE_FEATURE_NAMES}
43                 person = {fn: 1.0 for fn in fusion_audio.PERSON_FEATURE_NAMES}
44                 audio = {fn: 0.0 for fn in af.AUDIO_FEATURE_NAMES}
45                 audio["audio_hit_count"] = 8.0 if pos else 0.0
46                 cand.append({"start": s, "end": e, "shuttle": shuttle, "person": person,
47                             "audio": audio, "label": pos})
48             data = {"name": name, "fps": 30.0, "frame_width": 1920.0,
49                    "candidates": cand, "gts": [[0.0, 10.0], [20.0, 30.0]]}
50             with open(os.path.join(d, f"{name}_golden_features.json"), "w", encoding="utf-8") as
f:
51                 json.dump(data, f)
52
53
54         def test_audio_increment_compare(tmp_path):
55             d = str(tmp_path)
56             for k in range(6):
57                 _write_video(d, f"v{k}")
58             videos = fusion_audio.load_videos_with_audio(d)
59             assert len(videos) == 6
60             res = fusion_audio.compare_increment(videos, iou=0.5)
61             assert res["variant_b"]["honest_lovo"] is True
62             assert res["delta"]["f1"] > 0.0                # audio adds separating signal on top of
constant shuttle+person
63             assert "audio" in fusion_audio.format_result(res, 0.5).lower()
64
65
66         def test_load_videos_requires_audio(tmp_path):
67             data = {"name": "x", "fps": 30.0, "frame_width": 1920.0,
68                    "candidates": [{"start": 0.0, "end": 1.0,
69                                "shuttle": {fn: 0.0 for fn in
fusion_audio.SHUTTLE_FEATURE_NAMES},
70                                "person": {fn: 0.0 for fn in
fusion_audio.PERSON_FEATURE_NAMES},
71                                "label": 0}],
72                    "gts": [[0.0, 1.0]]}
73             with open(os.path.join(str(tmp_path), "x_golden_features.json"), "w",
encoding="utf-8") as f:
74                 json.dump(data, f)
75             with pytest.raises(SystemExit):
76                 fusion_audio.load_videos_with_audio(str(tmp_path))
77

```

16. assistant (2026-06-18T01:08:34.916Z)

Now I understand the full picture. Plan:

- Add a module-level `logger` + a self-contained `\_is\_fusion\_schema` predicate to `fusion\_compare.py` (its natural home ? `fusion\_audio` already imports from it). I'll ****duplicate**** rather than import from `ablation.py` because that fix is unmerged on master; importing would stack on it.
- `fusion\_compare.load\_videos`: skip non-fusion files with an info log.

- ``fusion_audio.load_videos_with_audio``: import the predicate, skip windowing-only files
- **before**** the audio-missing check (so genuine fusion-without-audio still raises).
- Add a regression test per loader.

Let me set up an isolated worktree off ``origin/master``.

17. user (2026-06-18T01:08:37.160Z)

Preparing worktree (new branch 'fix/fusion-loaders-mixed-schema')
branch 'fix/fusion-loaders-mixed-schema' set up to track 'origin/master'.
HEAD is now at be39a70 docs: rally-detection quality report ? technical-paper checkpoint (2026-06-17) (#206)
=== worktree HEAD ===
be39a70 docs: rally-detection quality report ? technical-paper checkpoint (2026-06-17) (#206)
fix/fusion-loaders-mixed-schema

18. user (2026-06-18T01:08:44.244Z)

```
=== fusion_compare diff (worktree vs what I read) ===
1,102c1,102
< """Pure-CPU LOVO comparison: shuttle-only vs shuttle+person learned fusion on the golden set.
<
< < The fusion-P3 measurement. Loads the per-video feature JSONs written by `fusion_golden.py`
< < and runs the SHIPPED experiment harness (`experiment.compare`, honest leave-one-video-out) ?
< < no new scoring/labeling math. The two variants differ ONLY by `feature_names` (the harness's
< < designed seam): the person features are extra columns the learned LogisticRegression weights,
< < so the held-out ?F1 is an honest answer to "does the person cue help?" ? not a hunch.
<
< < Run (no GPU):
< < python -m backend.eval.fusion_compare --features-dir output --record
< < """
< < from __future__ import annotations
<
< < import argparse
< < import glob
< < import json
< < import os
< < from typing import Any, Dict, List
<
< < from backend.eval import experiment
< < from backend.eval.eval_stats import paired_delta_ci
< < from backend.eval.fusion_golden import PERSON_FEATURE_NAMES, SHUTTLE_FEATURE_NAMES
<
<
< < def delta_stats(res: Dict[str, Any], metric: str = "f1") -> Dict[str, Any]:
< < """Paired (by video) B?A summary for one metric: mean delta + bootstrap CI + sign test
< < (eval_stats, C1 ? "never a bare mean"). Aligns the two variants' per-video held-out scores
< < by video name, so the headline ?F1 comes with how-wide + did-it-win-on-enough-videos."""
< < by_a = {p["video"]: p[metric] for p in res["variant_a"]["per_video"]}
< < by_b = {p["video"]: p[metric] for p in res["variant_b"]["per_video"]}
< < vids = [p["video"] for p in res["variant_a"]["per_video"]]
< < return paired_delta_ci([by_a[v] for v in vids], [by_b[v] for v in vids])
<
<
< < def load_videos(features_dir: str) -> List[experiment.VideoCandidates]:
< < """Load every ``*_golden_features.json`` into a VideoCandidates (intervals + the 10
< < feature columns + golden gts). The harness derives labels from gts itself."""
< < videos: List[experiment.VideoCandidates] = []
< < for path in sorted(glob.glob(os.path.join(features_dir, "*_golden_features.json"))):
< <     with open(path, encoding="utf-8") as f:
< <         d = json.load(f)
< <         cands = d["candidates"]
< <         intervals = [(float(c["start"]), float(c["end"])) for c in cands]
< <         features: Dict[str, List[float]] = {}
< <         for fn in SHUTTLE_FEATURE_NAMES:
```

```

<         features[fn] = [float(c["shuttle"][fn]) for c in candls]
<     for fn in PERSON_FEATURE_NAMES:
<         features[fn] = [float(c["person"][fn]) for c in candls]
<     gts = [(float(a), float(b)) for a, b in d["gts"]]
<     videos.append(experiment.VideoCandidates(name=d["name"], intervals=intervals,
<                                         features=features, gts=gts))
<     return videos
<
<
< def compare(videos: List[experiment.VideoCandidates], iou: float = 0.5,
<             threshold: float = 0.5) -> Dict[str, Any]:
<     shuttle_only = experiment.Variant(name="shuttle_only",
feature_names=list(SHUTTLE_FEATURE_NAMES),
<                                     threshold=threshold)
<     fusion = experiment.Variant(name="shuttle_person_fusion",
<                                 feature_names=list(SHUTTLE_FEATURE_NAMES) +
list(PERSON_FEATURE_NAMES),
<                                     threshold=threshold)
<     return experiment.compare(videos, shuttle_only, fusion, iou=iou, honest=True)
<
<
< def format_result(res: Dict[str, Any], iou: float) -> str:
<     a, b, d = res["variant_a"]["aggregate"], res["variant_b"]["aggregate"], res["delta"]
<     ds = delta_stats(res)
<     st = ds["sign_test"]
<     lines = [
<         f"=== Fusion P3 LOVO (n={res['num_videos']} golden videos, IoU>={iou}, honest) ===",
<         f"  shuttle-only : F1={a['f1']:.3f}  P={a['precision']:.3f}  R={a['recall']:.3f}
IoU={a['mean_iou']:.3f}",
<         f"    +person      : F1={b['f1']:.3f}  P={b['precision']:.3f}  R={b['recall']:.3f}
IoU={b['mean_iou']:.3f}",
<         f"    ?F1 = {d['f1']:+.3f}  ({res['delta_pct']['f1']:+.1f}%)  ?P={d['precision']:+.3f}
?R={d['recall']:+.3f}",
<         f"    paired ?F1 {ds['mean_delta']:+.3f}  [95% CI {ds['ci_low']:+.3f},
{ds['ci_high']:+.3f}]  "
<         f"| sign test {int(st['wins'])}W/{int(st['losses'])}L (p={st['p_value']:.3f})  "
<         f"| CI clears 0: {ds['ci_excludes_zero']}",
<         "  per-video ?F1:",
<     ]
<     for pv in res["per_video_delta"]:
<         lines.append(f"      {pv['video']}<26s ?F1={pv['f1']:+.3f}")
<     return "\n".join(lines)
<
<
< def main() -> None:
<     ap = argparse.ArgumentParser(description="LOVO fusion lift: shuttle-only vs shuttle+person
(golden set).")
<     ap.add_argument("--features-dir", default="output")
<     ap.add_argument("--iou", type=float, default=0.5)
<     ap.add_argument("--record", action="store_true", help="append the result to the experiment
leaderboard")
<     args = ap.parse_args()
<
<     videos = load_videos(args.features_dir)
<     if len(videos) < 2:
<         raise SystemExit(f"need >=2 *_golden_features.json in {args.features_dir} (found
{len(videos)})")
<     res = compare(videos, iou=args.iou)
<     print(format_result(res, args.iou))
<     if args.record:
<         row = experiment.record_experiment(res)
<         print(f"recorded -> eval_baselines/experiment_leaderboard.json ({row['timestamp']}")
<
<
< if __name__ == "__main__":

```



```

>     return experiment.compare(videos, shuttle_only, fusion, iou=iou, honest=True)
>
>
> def format_result(res: Dict[str, Any], iou: float) -> str:
>     a, b, d = res["variant_a"]["aggregate"], res["variant_b"]["aggregate"], res["delta"]
>     ds = delta_stats(res)
>     st = ds["sign_test"]
>     lines = [
>         f"=== Fusion P3 LOVO (n={res['num_videos']} golden videos, IoU>={iou}, honest) ===",
>         f"  shuttle-only : F1={a['f1']:.3f}  P={a['precision']:.3f}  R={a['recall']:.3f}
IoU={a['mean_iou']:.3f}",
>         f"  +person      : F1={b['f1']:.3f}  P={b['precision']:.3f}  R={b['recall']:.3f}
IoU={b['mean_iou']:.3f}",
>         f"    ?F1 = {d['f1']:+.3f}  ({res['delta_pct']['f1']:+.1f}%)    ?P={d['precision']:+.3f}
?R={d['recall']:+.3f}",
>         f"    paired ?F1 {ds['mean_delta']:+.3f}  [95% CI {ds['ci_low']:+.3f},
{ds['ci_high']:+.3f}]  "
>         f"| sign test {int(st['wins'])}W/{int(st['losses'])}L (p={st['p_value']:.3f})  "
>         f"| CI clears 0: {ds['ci_excludes_zero']}",
>         "  per-video ?F1:",
>     ]
>     for pv in res["per_video_delta"]:
>         lines.append(f"      {pv['video']:<26s} ?F1={pv['f1']:+.3f}")
>     return "\n".join(lines)
>
>
> def main() -> None:
>     ap = argparse.ArgumentParser(description="LOVO fusion lift: shuttle-only vs shuttle+person
(golden set).")
>     ap.add_argument("--features-dir", default="output")
>     ap.add_argument("--iou", type=float, default=0.5)
>     ap.add_argument("--record", action="store_true", help="append the result to the experiment
leaderboard")
>     args = ap.parse_args()
>
>     videos = load_videos(args.features_dir)
>     if len(videos) < 2:
>         raise SystemExit(f"need >=2 *_golden_features.json in {args.features_dir} (found
{len(videos)})")
>     res = compare(videos, iou=args.iou)
>     print(format_result(res, args.iou))
>     if args.record:
>         row = experiment.record_experiment(res)
>         print(f"recorded -> eval_baselines/experiment_leaderboard.json ({row['timestamp']}")
>
>
> if __name__ == "__main__":
>     main()
=== confirm_is_fusion_schema absent in worktree ===
absent (as expected ? ablation fix unmerged)

```

19. assistant (2026-06-18T01:08:55.637Z)

Content is identical (the diff is just CRLF vs LF display). Let me check line-ending config so my edits don't introduce noise.

20. user (2026-06-18T01:08:57.966Z)

=== .gitattributes ===

Golden Regression Fixtures (M1) ? pin LF line endings so the committed fixtures are byte-stable across OSes despite core.autocrlf. The characterization gate parse-comp (never byte-compares) JSON, but a stable on-disk form keeps `git diff` reviewable and

fixtures reproducible. See docs/GOLDEN_REGRESSION_FIXTURES.md §4d/§4e.

```
eval_baselines/fixtures/** text eol=lf
=== autocrlf ===
true
=== file CRLF check ===
backend/eval/fusion_compare.py: Python script, Unicode text, UTF-8 text executable, with CRLF
line terminators
i/lf w/crlf attr/ backend/eval/fusion_audio.py
i/lf w/crlf attr/ backend/eval/fusion_compare.py
Shell cwd was reset to C:\Users\avidu\Projects\badminton-highlight-indexer
```

21. assistant (2026-06-18T01:09:07.952Z)

`i/lf w/crlf` with `autocrlf=true` ? git stores LF, so commits stay clean. Now let me read the
worktree copies (required before editing them) and make the changes.

22. user (2026-06-18T01:09:08.582Z)

```
1      """Pure-CPU LOVO comparison: shuttle-only vs shuttle+person learned fusion on the golden
set.
2
3      The fusion-P3 measurement. Loads the per-video feature JSONs written by
`fusion_golden.py`
4      and runs the SHIPPED experiment harness (`experiment.compare`, honest leave-one-video-
out) ?
5      no new scoring/labeling math. The two variants differ ONLY by `feature_names` (the
harness's
6      designed seam): the person features are extra columns the learned LogisticRegression
weights,
7      so the held-out ?F1 is an honest answer to "does the person cue help?" ? not a hunch.
8
9      Run (no GPU):
10     python -m backend.eval.fusion_compare --features-dir output --record
11     """
12     from __future__ import annotations
13
14     import argparse
15     import glob
16     import json
17     import os
18     from typing import Any, Dict, List
19
20     from backend.eval import experiment
21     from backend.eval.eval_stats import paired_delta_ci
22     from backend.eval.fusion_golden import PERSON_FEATURE_NAMES, SHUTTLE_FEATURE_NAMES
23
24
25     def delta_stats(res: Dict[str, Any], metric: str = "f1") -> Dict[str, Any]:
26         """Paired (by video) B?A summary for one metric: mean delta + bootstrap CI + sign
test
27         (eval_stats, C1 ? "never a bare mean"). Aligns the two variants' per-video held-out
scores
28         by video name, so the headline ?F1 comes with how-wide + did-it-win-on-enough-
videos."""
29         by_a = {p["video"]: p[metric] for p in res["variant_a"]["per_video"]}
30         by_b = {p["video"]: p[metric] for p in res["variant_b"]["per_video"]}
31         vids = [p["video"] for p in res["variant_a"]["per_video"]]
32         return paired_delta_ci([by_a[v] for v in vids], [by_b[v] for v in vids])
33
34
35     def load_videos(features_dir: str) -> List[experiment.VideoCandidates]:
36         """Load every ``*_golden_features.json`` into a VideoCandidates (intervals + the 10
37         feature columns + golden gts). The harness derives labels from gts itself."""
38         videos: List[experiment.VideoCandidates] = []
```

```

39         for path in sorted(glob.glob(os.path.join(features_dir, "*_golden_features.json"))):
40             with open(path, encoding="utf-8") as f:
41                 d = json.load(f)
42                 cands = d["candidates"]
43                 intervals = [(float(c["start"]), float(c["end"])) for c in cands]
44                 features: Dict[str, List[float]] = {}
45                 for fn in SHUTTLE_FEATURE_NAMES:
46                     features[fn] = [float(c["shuttle"][fn]) for c in cands]
47                 for fn in PERSON_FEATURE_NAMES:
48                     features[fn] = [float(c["person"][fn]) for c in cands]
49                 gts = [(float(a), float(b)) for a, b in d["gts"]]
50                 videos.append(experiment.VideoCandidates(name=d["name"], intervals=intervals,
51                                                         features=features, gts=gts))
52     return videos

```

23. user (2026-06-18T01:09:14.693Z)

The file C:\Users\avidu\Projects\bhi-fusion-loaders\backend\eval\fusion_compare.py has been updated successfully. (file state is current in your context ? no need to Read it back)

24. assistant (2026-06-18T01:09:11.799Z)

Now the edits to `fusion\_compare.py`. First, imports + logger:

25. assistant (2026-06-18T01:09:17.706Z)

Now add the predicate and update `load\_videos`:

26. user (2026-06-18T01:09:27.970Z)

The file C:\Users\avidu\Projects\bhi-fusion-loaders\backend\eval\fusion_compare.py has been updated successfully. (file state is current in your context ? no need to Read it back)

27. assistant (2026-06-18T01:09:30.802Z)

Now `fusion\_audio.py`. Read the worktree copy's header, then edit:

28. user (2026-06-18T01:09:31.319Z)

```

1     """Fusion P4 ? measure the INCREMENTAL audio-cue lift on top of the person cue (CPU-
2     only).
3     Reuses the person-feature JSONs from `fusion_golden.py` (no GPU re-detect): `augment`
4     runs
5     `detect_hits_in_video` once per video (ffmpeg + numpy, CPU) and adds the 3 audio
6     features per
7     window; `compare_increment` then runs the SHIPPED harness between a shuttle+person
8     variant
9     and a shuttle+person+audio variant ? so audio is measured as a marginal contribution ON
10    TOP
11    of person, not against raw shuttle. Audio enters only as columns the learned scorer
12    weights
13    (no special-casing, default-OFF); the golden-set ?F1 decides whether it earns its keep.
14
15    Run (no GPU):
16    python -m backend.eval.fusion_audio --manifest output/human_lovo_manifest.json
17    --features-dir output --augment --compare
18    """
19    from __future__ import annotations
20
21    import argparse
22    import glob
23    import json
24    import os

```

```

19     from typing import Any, Dict, List
20
21     from backend.eval import experiment
22     from backend.eval.fusion_compare import delta_stats
23     from backend.eval.fusion_golden import PERSON_FEATURE_NAMES, SHUTTLE_FEATURE_NAMES,
_derive_video_path
24     from backend.pipeline.detectors import audio_features as af
25     from backend.pipeline.audio.hit_detector import detect_hits_in_video
26

```

29. user (2026-06-18T01:09:35.898Z)

The file C:\Users\avidu\Projects\bhi-fusion-loaders\backend\eval\fusion_audio.py has been updated successfully. (file state is current in your context ? no need to Read it back)

30. assistant (2026-06-18T01:09:40.986Z)

Now add a logger to `fusion\_audio` and update `load\_videos\_with\_audio`. First the import/logger:

31. user (2026-06-18T01:09:44.815Z)

The file C:\Users\avidu\Projects\bhi-fusion-loaders\backend\eval\fusion_audio.py has been updated successfully. (file state is current in your context ? no need to Read it back)

32. user (2026-06-18T01:09:54.803Z)

The file C:\Users\avidu\Projects\bhi-fusion-loaders\backend\eval\fusion_audio.py has been updated successfully. (file state is current in your context ? no need to Read it back)

33. assistant (2026-06-18T01:10:03.201Z)

Now the regression tests. Let me read the worktree test files and add a test to each.

34. user (2026-06-18T01:10:03.681Z)

```

1     """GPU-free tests for the fusion-P3 measurement (fusion_golden extractor +
fusion_compare LOVO).
2
3     The LOVO driver is exercised on a synthetic 6-video set where the SHUTTLE features are
4     non-separating (constant) but a PERSON feature cleanly separates rallies ? so the
harness must
5     report a positive fusion ?F1. The extractor is exercised with a stubbed detector (no
GPU).
6     """
7     import json
8     import os
9     from types import SimpleNamespace
10
11     import pytest
12
13     from backend.eval import fusion_compare
14     from backend.eval.fusion_compare import PERSON_FEATURE_NAMES, SHUTTLE_FEATURE_NAMES
15
16
17     def _write_video(d, name):
18         """A video whose shuttle features are identical for pos/neg (no signal) but whose
19         players_in_court separates rallies (4) from junk (0)."""
20         cands = []
21         for (s, e), pos in [((0.0, 10.0), 1), ((20.0, 30.0), 1), ((40.0, 41.0), 0), ((50.0,
51.0), 0)]:
22             shuttle = {fn: 1.0 for fn in SHUTTLE_FEATURE_NAMES}          # constant
-> non-separating
23             person = {fn: 0.0 for fn in PERSON_FEATURE_NAMES}
24             person["players_in_court"] = 4.0 if pos else 0.0              # the

```

```

separating signal
25         person["in_court_ratio"] = 1.0 if pos else 0.0
26         cands.append({"start": s, "end": e, "shuttle": shuttle, "person": person,
"label": pos})
27         data = {"name": name, "fps": 30.0, "frame_width": 1920.0,
28                 "candidates": cands, "gts": [[0.0, 10.0], [20.0, 30.0]]}
29         with open(os.path.join(d, f"{name}_golden_features.json"), "w", encoding="utf-8") as
f:
30             json.dump(data, f)
31
32
33     def test_fusion_compare_reports_person_driven_lift(tmp_path):
34         d = str(tmp_path)
35         for k in range(6):
36             _write_video(d, f"vid{k}")
37         videos = fusion_compare.load_videos(d)
38         assert len(videos) == 6
39         res = fusion_compare.compare(videos, iou=0.5)
40         assert res["num_videos"] == 6
41         assert res["variant_b"]["honest_lovo"] is True # LOVO-honest learned
variant
42         # Fusion (with the separating person feature) must beat shuttle-only (no signal).
43         assert res["delta"]["f1"] > 0.0
44         assert res["variant_b"]["aggregate"]["f1"] > res["variant_a"]["aggregate"]["f1"]
45         # format_result must not crash and must mention both variants.
46         out = fusion_compare.format_result(res, 0.5)
47         assert "shuttle-only" in out and "+person" in out
48
49
50     def test_fusion_compare_delta_stats_and_ci(tmp_path):
51         """delta_stats yields a paired bootstrap CI + sign test; format_result reports
them."""
52         d = str(tmp_path)
53         for k in range(6):
54             _write_video(d, f"vid{k}")
55         res = fusion_compare.compare(fusion_compare.load_videos(d), iou=0.5)
56         ds = fusion_compare.delta_stats(res)
57         assert set(ds) >= {"mean_delta", "ci_low", "ci_high", "sign_test",
"ci_excludes_zero"}
58         assert ds["ci_low"] - 1e-9 <= ds["mean_delta"] <= ds["ci_high"] + 1e-9 # CI
brackets mean (±fp)
59         assert ds["sign_test"]["wins"] == 6 # fusion wins on every video (person
feature separates)
60         assert ds["ci_excludes_zero"] is True
61         out = fusion_compare.format_result(res, 0.5)
62         assert "CI" in out and "sign test" in out
63
64
65     def test_fusion_compare_needs_two_videos(tmp_path):
66         _write_video(str(tmp_path), "only")
67         videos = fusion_compare.load_videos(str(tmp_path))
68         assert len(videos) == 1 # compare() / experiment.compare needs >=2 for honest LOVO
69

```

35. user (2026-06-18T01:10:14.649Z)

The file C:\Users\avidu\Projects\bhi-fusion-loaders\tests\test_fusion_compare.py has been updated successfully. (file state is current in your context ? no need to Read it back)

36. assistant (2026-06-18T01:10:18.323Z)

Now the audio test:

37. user (2026-06-18T01:10:19.303Z)

```

36      # ----- incremental-lift driver
(synthetic)
37
38      def _write_video(d, name):
39          """shuttle + person constant (no signal); audio_hit_count separates rallies (8) from
junk (0)."""
40          cand = []
41          for (s, e), pos in [((0.0, 10.0), 1), ((20.0, 30.0), 1), ((40.0, 41.0), 0), ((50.0,
51.0), 0)]:
42              shuttle = {fn: 1.0 for fn in fusion_audio.SHUTTLE_FEATURE_NAMES}
43              person = {fn: 1.0 for fn in fusion_audio.PERSON_FEATURE_NAMES}
44              audio = {fn: 0.0 for fn in af.AUDIO_FEATURE_NAMES}
45              audio["audio_hit_count"] = 8.0 if pos else 0.0
46              cand.append({"start": s, "end": e, "shuttle": shuttle, "person": person,
47                          "audio": audio, "label": pos})
48          data = {"name": name, "fps": 30.0, "frame_width": 1920.0,
49                "candidates": cand, "gts": [[0.0, 10.0], [20.0, 30.0]]}
50          with open(os.path.join(d, f"{name}_golden_features.json"), "w", encoding="utf-8") as
f:
51              json.dump(data, f)
52
53
54      def test_audio_increment_compare(tmp_path):
55          d = str(tmp_path)
56          for k in range(6):
57              _write_video(d, f"v{k}")
58          videos = fusion_audio.load_videos_with_audio(d)
59          assert len(videos) == 6
60          res = fusion_audio.compare_increment(videos, iou=0.5)
61          assert res["variant_b"]["honest_lovo"] is True
62          assert res["delta"]["f1"] > 0.0          # audio adds separating signal on top of
constant shuttle+person
63          assert "audio" in fusion_audio.format_result(res, 0.5).lower()
64
65
66      def test_load_videos_requires_audio(tmp_path):
67          data = {"name": "x", "fps": 30.0, "frame_width": 1920.0,
68                "candidates": [{"start": 0.0, "end": 1.0,
69                              "shuttle": {fn: 0.0 for fn in
fusion_audio.SHUTTLE_FEATURE_NAMES},
70                              "person": {fn: 0.0 for fn in
fusion_audio.PERSON_FEATURE_NAMES},
71                              "label": 0}],
72                "gts": [[0.0, 1.0]]}
73          with open(os.path.join(str(tmp_path), "x_golden_features.json"), "w",
encoding="utf-8") as f:
74              json.dump(data, f)
75          with pytest.raises(SystemExit):
76              fusion_audio.load_videos_with_audio(str(tmp_path))
77

```

38. user (2026-06-18T01:10:33.934Z)

The file C:\Users\avidu\Projects\bhi-fusion-loaders\tests\test_audio_features.py has been updated successfully. (file state is current in your context ? no need to Read it back)

39. assistant (2026-06-18T01:10:37.300Z)

Let me verify `VideoCandidates` exposes `.name`, then run the targeted tests in the worktree.

40. user (2026-06-18T01:10:38.483Z)

```

58:logger = logging.getLogger(__name__)
100:      - ``feature_names`` ? the persisted features the learned filter consumes. When set

```

```

107:     name: str
114:     feature_names: Optional[List[str]] = None
125:     return self.classifier_model is not None or bool(self.feature_names)
129:         "name": self.name,
136:         "feature_names": self.feature_names,
158: CONTROL = Variant(name="control")
Shell cwd was reset to C:\Users\avidu\Projects\badminton-highlight-indexer

```

41. user (2026-06-18T01:10:45.061Z)

```

165: class VideoCandidates:
class VideoCandidates:
    """One video's persisted detection, ready for offline re-scoring.

    ``intervals`` are the candidate windows; ``features`` is column-major
    (``feature_name -> per-candidate value``, each list aligned to ``intervals``);
    ``gts`` are golden labels (None for new/unlabeled footage). Build one from the DB
    with ``load_video_candidates``, or directly in tests.
    """

    name: str
    intervals: List[Interval]
    features: Dict[str, List[float]] = field(default_factory=dict)
    gts: Optional[List[Interval]] = None

def _feature_column(vc: VideoCandidates, name: str) -> List[float]:
Shell cwd was reset to C:\Users\avidu\Projects\badminton-highlight-indexer

```

42. assistant (2026-06-18T01:10:47.536Z)

``VideoCandidates.name`` exists. Now run the two targeted test files in the worktree:

43. user (2026-06-18T01:10:53.755Z)

```

..... [100%]
14 passed in 4.09s
Shell cwd was reset to C:\Users\avidu\Projects\badminton-highlight-indexer

```

44. assistant (2026-06-18T01:10:59.729Z)

All 14 pass (was 12). Now the full verification gate. Let me check the repo's CI config for the exact lint/type/coverage commands.

45. user (2026-06-18T01:11:01.427Z)

```

=== pytest/cov config ===
=== mypy targets in CI ===
16: # Lint gate (ruff.toml at repo root): pyflakes/pycodestyle + bugbear.
17: # Also the repo-wide syntax guard ? ruff reports invalid-syntax on every
28:     - name: Install ruff
29:         run: python -m pip install ruff
31:         run: ruff check . --output-format github
33: # Type gate (mypy.ini at repo root): lenient green baseline with an explicit
35: # up. Typed core deps are installed so mypy sees their real signatures
45:     - name: Install mypy + typed core deps
46:         run: python -m pip install mypy pydantic fastapi uvicorn python-dotenv google-genai
numpy
48:         run: mypy backend training
Shell cwd was reset to C:\Users\avidu\Projects\badminton-highlight-indexer

```

46. user (2026-06-18T01:11:07.104Z)

```

=== run_all_tests.py head ===

```

```

4:A single, discoverable entry point so you (or a reviewer / CI) can verify the
8:   python run_all_tests.py -v           # verbose (passthrough to pytest)
10:   python run_all_tests.py --cov=backend --cov-report=term-missing # coverage
12:Anything after the script name is passed straight through to pytest.
18: package isn't installed (they use ``pytest.importorskip``). To exercise them too:
20:- Exit code is pytest's own, so this is CI / pre-PR friendly (0 = all green).
29:def _installed(module: str) -> bool:
36:def main() -> int:
41:   cmd = [sys.executable, "-m", "pytest", *args]
53:   print(" RESULT: " + ("PASS - all tests green" if rc == 0 else f"FAIL - pytest exit
{rc}"))
58:if __name__ == "__main__":
59:   raise SystemExit(main())
=== cov-fail-under anywhere ===
./.github/workflows/ci.yml:97:   run: python run_all_tests.py --cov=backend --cov=training
--cov-report=term --cov-fail-under=70
Shell cwd was reset to C:\Users\avidu\Projects\badminton-highlight-indexer

```

47. assistant (2026-06-18T01:11:12.665Z)

CI gate is ``--cov-fail-under=70`` on ``backend`` + ``training``. Let me run the full suite with coverage, plus ruff and mypy.

48. user (2026-06-18T01:13:02.263Z)

backend\providers__init__.py	16	1	94%
backend\providers\base.py	9	1	89%
backend\providers\gemini.py	98	19	81%
backend\reporting__init__.py	26	2	92%
backend\reporting\harvest.py	111	8	93%
backend\reporting\run_signals.py	14	0	100%
backend\results.py	22	0	100%
backend\sports__init__.py	17	1	94%
backend\sports\badminton.py	13	1	92%
backend\sports\base.py	12	3	75%
backend\tools__init__.py	0	0	100%
backend\tools\export.py	75	2	97%
backend\tools\rally_slicer.py	104	47	55%
backend\utils__init__.py	0	0	100%
backend\utils\geometry.py	29	0	100%
backend\utils\hashing.py	9	0	100%
backend\utils\logging_setup.py	11	0	100%
backend\utils\paths.py	27	3	89%
backend\utils\run_telemetry.py	126	12	90%
backend\utils\video.py	25	3	88%
training__init__.py	0	0	100%
training\gen0__init__.py	0	0	100%
training\gen0\harness.py	108	11	90%

TOTAL	7992	1546	81%

Required test coverage of 70% reached. Total coverage: 80.66%

846 passed, 7 skipped, 7 warnings in 103.74s (0:01:43)

=====

RESULT: PASS - all tests green

=====

Shell cwd was reset to C:\Users\avidu\Projects\badminton-highlight-indexer

49. assistant (2026-06-18T01:13:05.925Z)

Full suite green: ****846 passed, 7 skipped****, coverage ****80.66%**** (?70). Now ruff + mypy:

50. user (2026-06-18T01:14:02.574Z)

=== RUFF ===

All checks passed!

=== MYPY ===

```
backend\annotations\base.py:85: note: By default the bodies of untyped functions are not
checked, consider using --check-untyped-defs [annotation-unchecked]
backend\sports\__init__.py:7: note: By default the bodies of untyped functions are not checked,
consider using --check-untyped-defs [annotation-unchecked]
backend\pipeline\validators\base.py:36: note: By default the bodies of untyped functions are not
checked, consider using --check-untyped-defs [annotation-unchecked]
backend\providers\__init__.py:7: note: By default the bodies of untyped functions are not
checked, consider using --check-untyped-defs [annotation-unchecked]
Success: no issues found in 104 source files
Shell cwd was reset to C:\Users\avidu\Projects\badminton-highlight-indexer
```

51. assistant (2026-06-18T01:14:08.192Z)

All gates green ? suite 846 passed, coverage 80.66%, ruff clean, mypy clean (only informational notes). Let me review the complete diff before committing.

52. user (2026-06-18T01:14:09.483Z)

```
backend/eval/fusion_audio.py | 21 ++++++-----
backend/eval/fusion_compare.py | 35 ++++++-----
tests/test_audio_features.py | 26 ++++++-----
tests/test_fusion_compare.py | 25 ++++++-----
4 files changed, 100 insertions(+), 7 deletions(-)
=== FULL DIFF ===
diff --git a/backend/eval/fusion_audio.py b/backend/eval/fusion_audio.py
index 8290614..d59dd46 100644
--- a/backend/eval/fusion_audio.py
+++ b/backend/eval/fusion_audio.py
@@ -15,15 +15,18 @@ from __future__ import annotations
 import argparse
 import glob
 import json
+import logging
 import os
 from typing import Any, Dict, List

 from backend.eval import experiment
-from backend.eval.fusion_compare import delta_stats
+from backend.eval.fusion_compare import _is_fusion_schema, delta_stats
 from backend.eval.fusion_golden import PERSON_FEATURE_NAMES, SHUTTLE_FEATURE_NAMES,
 _derive_video_path
 from backend.pipeline.detectors import audio_features as af
 from backend.pipeline.audio.hit_detector import detect_hits_in_video

+logger = logging.getLogger(__name__)
+
def augment(manifest_path: str, features_dir: str, k: float = 2.5) -> List[str]:
    """Add the 3 audio features to each existing ``<name>_golden_features.json`` window."""
@@ -50,14 +53,24 @@ def augment(manifest_path: str, features_dir: str, k: float = 2.5) ->
List[str]:

    def load_videos_with_audio(features_dir: str) -> List[experiment.VideoCandidates]:
-        """Load the feature JSONs into VideoCandidates including the audio columns (requires
-        `augment` to have run ? raises if a JSON lacks audio)."""
+        """Load the FULL-FUSION-SCHEMA feature JSONs into VideoCandidates including the audio
+        columns (requires `augment` to have run ? raises if a fusion JSON lacks audio).
+
+        Leaner velocity-windowing-only files (no ``shuttle``/``person`` cue dicts) are SKIPPED, not
+        crashed on, so a mixed corpus loads the fusion subset cleanly (see ``_is_fusion_schema``).
+        The skip is checked BEFORE the audio-presence guard, so a genuine fusion file that simply
+        hasn't been ``--augment``-ed still raises rather than being silently dropped."""
```



```

videos: List[experiment.VideoCandidates] = []
for path in sorted(glob.glob(os.path.join(features_dir, "*_golden_features.json"))):
    with open(path, encoding="utf-8") as f:
        d = json.load(f)
        cand = d["candidates"]
        if cand and "audio" not in cand[0]:
-         if not _is_fusion_schema(cand):
+             logger.info("fusion_audio.load_videos_with_audio: skipping %s ? windowing-only "
+                 "schema (no shuttle/person cue dicts), not a fusion-feature file",
+                 os.path.basename(path))
+             continue
+         if "audio" not in cand[0]:
            raise SystemExit(f"{path} has no audio features ? run `--augment` first")
            intervals = [(float(c["start"]), float(c["end"])) for c in cand]
            features: Dict[str, List[float]] = {}
diff --git a/backend/eval/fusion_compare.py b/backend/eval/fusion_compare.py
index 463ec7e..9372ba0 100644
--- a/backend/eval/fusion_compare.py
+++ b/backend/eval/fusion_compare.py
@@ -14,13 +14,16 @@ from __future__ import annotations
import argparse
import glob
import json
+import logging
import os
-from typing import Any, Dict, List
+from typing import Any, Dict, List, Sequence

from backend.eval import experiment
from backend.eval.eval_stats import paired_delta_ci
from backend.eval.fusion_golden import PERSON_FEATURE_NAMES, SHUTTLE_FEATURE_NAMES

+logger = logging.getLogger(__name__)
+

def delta_stats(res: Dict[str, Any], metric: str = "f1") -> Dict[str, Any]:
    """Paired (by video) B7A summary for one metric: mean delta + bootstrap CI + sign test
@@ -32,14 +35,40 @@ def delta_stats(res: Dict[str, Any], metric: str = "f1") -> Dict[str, Any]:
    return paired_delta_ci([by_a[v] for v in vids], [by_b[v] for v in vids])

+def _is_fusion_schema(candidates: Sequence[Dict[str, Any]]) -> bool:
+    """True iff every candidate row carries the per-cue feature dicts the fusion loaders read
+    (`shuttle` + `person`; `audio` stays optional ? it is added later by
+    `fusion_audio.augment` and older JSONs predate it).
+
+    The velocity-windowing / distill corpus-growth path persists a LEANER
+    `*_golden_features.json` row (`start`/`end` + velocity stats, no cue dicts) for clips
+    that never went through fusion feature extraction. The loaders SKIP those rather than crash
+    on `c["shuttle"]` ? so a MIXED corpus (some fusion files, some windowing-only) loads the
+    fusion subset cleanly. An empty candidate list is not a fusion file either.
+
+    (Kept local to the fusion loaders; `ablation.py` carries its own copy of this one-liner ?
+    the two modules are independent and the shared form would force an awkward import.)"""
+    return bool(candidates) and all("shuttle" in c and "person" in c for c in candidates)
+
def load_videos(features_dir: str) -> List[experiment.VideoCandidates]:
-    """Load every `*_golden_features.json` into a VideoCandidates (intervals + the 10
-    feature columns + golden gts). The harness derives labels from gts itself."""
+    """Load every FULL-FUSION-SCHEMA `*_golden_features.json` into a VideoCandidates
+    (intervals + the 10 feature columns + golden gts). The harness derives labels from gts
+    itself.
+
+    Leaner velocity-windowing-only files (no `shuttle`/`person` cue dicts ? the schema the

```

```

+ corpus-growth path persists) are SKIPPED, not crashed on, so a mixed corpus loads the
+ fusion subset cleanly (see ``_is_fusion_schema``)."""
+ videos: List[experiment.VideoCandidates] = []
+ for path in sorted(glob.glob(os.path.join(features_dir, "*_golden_features.json"))):
+     with open(path, encoding="utf-8") as f:
+         d = json.load(f)
+         cand = d["candidates"]
+         if not _is_fusion_schema(cand):
+             logger.info("fusion_compare.load_videos: skipping %s ? windowing-only schema "
+                 "(no shuttle/person cue dicts), not a fusion-feature file",
+                 os.path.basename(path))
+             continue
+         intervals = [(float(c["start"]), float(c["end"])) for c in cand]
+         features: Dict[str, List[float]] = {}
+         for fn in SHUTTLE_FEATURE_NAMES:
diff --git a/tests/test_audio_features.py b/tests/test_audio_features.py
index 6e04730..d6f9d4e 100644
--- a/tests/test_audio_features.py
+++ b/tests/test_audio_features.py
@@ -51,6 +51,19 @@ def _write_video(d, name):
     json.dump(data, f)

+def _write_windowing_video(d, name):
+    """A LEANER velocity-windowing/distill ``*_golden_features.json`` (no shuttle/person/audio
+    cue dicts) ? the schema the corpus-growth path persists. The audio loader must SKIP these,
+    not KeyError on ``c["shuttle"]`` nor raise the no-audio guard against them."""
+    cand = [{"active_frame_count": 120, "chunk_index": i, "start": float(i * 10),
+        "end": float(i * 10 + 8), "mean_velocity": 1.2, "peak_velocity": 3.4,
+        "track_id_count": 2} for i in range(3)]
+    data = {"name": name, "fps": 30.0, "frame_width": 1920.0,
+        "candidates": cand, "gts": [[0.0, 8.0]]}
+    with open(os.path.join(d, f"{name}_golden_features.json"), "w", encoding="utf-8") as f:
+        json.dump(data, f)
+
+def test_audio_increment_compare(tmp_path):
+    d = str(tmp_path)
+    for k in range(6):
@@ -63,6 +76,19 @@ def test_audio_increment_compare(tmp_path):
     assert "audio" in fusion_audio.format_result(res, 0.5).lower()

+def test_load_videos_with_audio_skips_windowing_only_in_mixed_corpus(tmp_path):
+    """Regression: a MIXED corpus (augmented fusion files + leaner windowing-only files) must
+    load ONLY the fusion files ? neither KeyError on ``c["shuttle"]`` nor SystemExit the
+    no-audio guard against the windowing-only files (the skip is checked BEFORE that guard)."""
+    d = str(tmp_path)
+    for k in range(2):
+        _write_video(d, f"fusion{k}")
+    for k in range(2):
+        _write_windowing_video(d, f>window{k}")
+    videos = fusion_audio.load_videos_with_audio(d)
+    assert {v.name for v in videos} == {"fusion0", "fusion1"} # windowing-only files skipped
+
+def test_load_videos_requires_audio(tmp_path):
+    data = {"name": "x", "fps": 30.0, "frame_width": 1920.0,
+        "candidates": [{"start": 0.0, "end": 1.0,
diff --git a/tests/test_fusion_compare.py b/tests/test_fusion_compare.py
index 063dffc..054f602 100644
--- a/tests/test_fusion_compare.py
+++ b/tests/test_fusion_compare.py
@@ -30,6 +30,31 @@ def _write_video(d, name):
     json.dump(data, f)

```

```

+def _write_windowing_video(d, name):
+    """A LEANER velocity-windowing/distill ``*_golden_features.json`` ? the schema the corpus-
+    growth path persists (no shuttle/person/audio cue dicts). The fusion loaders must SKIP
+    these
+    rather than KeyError on ``c["shuttle"]``."""
+    cand = [{"active_frame_count": 120, "chunk_index": i, "start": float(i * 10),
+            "end": float(i * 10 + 8), "mean_velocity": 1.2, "peak_velocity": 3.4,
+            "track_id_count": 2} for i in range(3)]
+    data = {"name": name, "fps": 30.0, "frame_width": 1920.0,
+            "candidates": cand, "gts": [[0.0, 8.0]]}
+    with open(os.path.join(d, f"{name}_golden_features.json"), "w", encoding="utf-8") as f:
+        json.dump(data, f)
+
+
+def test_load_videos_skips_windowing_only_in_mixed_corpus(tmp_path):
+    """Regression: a MIXED corpus (fusion files + leaner windowing-only files) must load ONLY
+    the fusion files, not KeyError on ``c["shuttle"]`` (the n=11 real-`output/` bug)."""
+    d = str(tmp_path)
+    for k in range(2):
+        _write_video(d, f"fusion{k}")
+    for k in range(2):
+        _write_windowing_video(d, f"window{k}")
+    videos = fusion_compare.load_videos(d)
+    assert {v.name for v in videos} == {"fusion0", "fusion1"} # windowing-only files skipped
+
+
+def test_fusion_compare_reports_person_driven_lift(tmp_path):
+    d = str(tmp_path)
+    for k in range(6):

```

Shell cwd was reset to C:\Users\avidu\Projects\badminton-highlight-indexer

53. assistant (2026-06-18T01:14:22.368Z)

The diff is clean. Let me prove the regression tests genuinely catch the bug ? temporarily stash only the production fixes and confirm the new tests fail against the old loaders.

54. user (2026-06-18T01:14:30.748Z)

--- production fix stashed; running new tests against OLD loaders ---

```

features_dir = 'C:\\Users\\avidu\\AppData\\Local\\Temp\\pytest-of-
avidu\\pytest-490\\test_load_videos_with_audio_sk0'

```

```

def load_videos_with_audio(features_dir: str) -> List[experiment.VideoCandidates]:
    """Load the feature JSONs into VideoCandidates including the audio columns (requires
    `augment` to have run ? raises if a JSON lacks audio)."""
    videos: List[experiment.VideoCandidates] = []
    for path in sorted(glob.glob(os.path.join(features_dir, "*_golden_features.json"))):
        with open(path, encoding="utf-8") as f:
            d = json.load(f)
            cand = d["candidates"]
            if cand and "audio" not in cand[0]:
>                 raise SystemExit(f"{path} has no audio features ? run `--augment` first")
E                 SystemExit: C:\Users\avidu\AppData\Local\Temp\pytest-of-
avidu\pytest-490\test_load_videos_with_audio_sk0\window0_golden_features.json has no audio
features ? run `--augment` first

```

backend\eval\fusion_audio.py:61: SystemExit

===== short test summary info =====

FAILED tests/test_fusion_compare.py::test_load_videos_skips_windowing_only_in_mixed_corpus

FAILED

tests/test_audio_features.py::test_load_videos_with_audio_skips_windowing_only_in_mixed_corpus

2 failed in 3.20s

--- restoring production fix ---

restored

Shell cwd was reset to C:\Users\avidu\Projects\badminton-highlight-indexer

55. assistant (2026-06-18T01:14:39.139Z)

Both regression tests fail without the fix and pass with it ? genuinely validated (the audio one even exposes the misleading "run --augment first" error the old code raised on windowing-only files). Now commit, following the shared-checkout safety protocol: re-verify HEAD, stage explicit paths only.

56. user (2026-06-18T01:14:41.117Z)

```
=== branch ===
fix/fusion-loaders-mixed-schema
=== HEAD ===
be39a70 docs: rally-detection quality report ? technical-paper checkpoint (2026-06-17) (#206)
=== commits vs origin/master (should be empty pre-commit) ===
=== status ===
M backend/eval/fusion_audio.py
M backend/eval/fusion_compare.py
M tests/test_audio_features.py
M tests/test_fusion_compare.py
Shell cwd was reset to C:\Users\avidu\Projects\badminton-highlight-indexer
```

57. assistant (2026-06-18T01:14:44.425Z)

Clean state ? HEAD is at `origin/master`, only my 4 files modified. Staging explicit paths and committing:

58. user (2026-06-18T01:14:52.391Z)

25a5a78 fix(eval): fusion loaders skip windowing-only files on a mixed golden corpus
Shell cwd was reset to C:\Users\avidu\Projects\badminton-highlight-indexer

59. assistant (2026-06-18T01:15:01.399Z)

Committed. Let me verify the commit is clean (single commit vs origin/master, LF line endings) and push, then open the PR.

60. user (2026-06-18T01:15:04.462Z)

```
=== commits vs origin/master (should be ONLY mine) ===
25a5a78 fix(eval): fusion loaders skip windowing-only files on a mixed golden corpus
=== CRLF check on committed blobs (should be i/lf) ===
i/lf   w/crlf  attr/          backend/eval/fusion_audio.py
i/lf   w/crlf  attr/          backend/eval/fusion_compare.py
i/lf   w/crlf  attr/          tests/test_audio_features.py
i/lf   w/crlf  attr/          tests/test_fusion_compare.py
=== any CR in committed diff? ===
0
0 CR ? LF clean
Shell cwd was reset to C:\Users\avidu\Projects\badminton-highlight-indexer
```

61. assistant (2026-06-18T01:15:07.155Z)

Single clean LF commit, only mine vs `origin/master`. Pushing and opening the PR:

62. user (2026-06-18T01:15:10.185Z)

```
remote:
remote: Create a pull request for 'fix/fusion-loaders-mixed-schema' on GitHub by visiting:
remote:   https://github.com/avidullu/badminton-highlight-indexer/pull/new/fix/fusion-
loaders-mixed-schema
remote:
```

branch 'fix/fusion-loaders-mixed-schema' set up to track 'origin/fix/fusion-loaders-mixed-schema'.

To <https://github.com/avidullu/badminton-highlight-indexer.git>

* [new branch] fix/fusion-loaders-mixed-schema -> fix/fusion-loaders-mixed-schema

Shell cwd was reset to C:\Users\avidu\Projects\badminton-highlight-indexer

63. user (2026-06-18T01:15:27.839Z)

<https://github.com/avidullu/badminton-highlight-indexer/pull/210>

Shell cwd was reset to C:\Users\avidu\Projects\badminton-highlight-indexer

64. assistant (2026-06-18T01:15:31.725Z)

PR opened: [#210](<https://github.com/avidullu/badminton-highlight-indexer/pull/210>). Now clean up the isolated worktree (branch is pushed, so it's safe to remove):

65. user (2026-06-18T01:15:33.054Z)

worktree removed

C:/Users/avidu/Projects/badminton-highlight-indexer

f5f5e52 [docs/golden-regression-fixtures-plan]

C:/Users/avidu/Projects/badminton-highlight-indexer/.claude/worktrees/ablation-schema-fix

59ae347 [fix/ablation-litmus-mixed-schema]

C:/Users/avidu/Projects/badminton-highlight-indexer/.claude/worktrees/agent-a771ff1d85e18f2ed

7e0881a [perf/wasb-fast-frames]

C:/Users/avidu/Projects/badminton-highlight-indexer/.claude/worktrees/bhi-execpolicy

5cb7bbf [bhi-execpolicy]

C:/Users/avidu/Projects/badminton-highlight-indexer/.claude/worktrees/boxhill17

8736e2d [run/boxhill17-highlights]

C:/Users/avidu/Projects/badminton-highlight-indexer/.claude/worktrees/docs-golden

c33cf1d [docs/golden-videos-tracker]

C:/Users/avidu/Projects/badminton-highlight-indexer/.claude/worktrees/docs-rally-quality

877ab1f [docs/rally-quality-research]

C:/Users/avidu/Projects/badminton-highlight-indexer/.claude/worktrees/docs-today

ce7eb47 [docs/quality-iterations-realfootage]

C:/Users/avidu/Projects/badminton-highlight-indexer/.claude/worktrees/hardchop

614325d [feat/w1-hard-cap-fallback]

C:/Users/avidu/Projects/badminton-highlight-indexer/.claude/worktrees/local-noai

3ea7ad4 [feat/local-no-ai-segmenter]

C:/Users/avidu/Projects/badminton-highlight-indexer/.claude/worktrees/pr-stitch

a6adcc5 [fix/runner-stitch-watermark]

C:/Users/avidu/Projects/badminton-highlight-indexer/.claude/worktrees/r2-impl

a450c53 [feat/r2-tolerance-metric]

C:/Users/avidu/Projects/badminton-highlight-indexer/.claude/worktrees/r4-endrule

e248d2d [feat/r4-inactivity-end]

C:/Users/avidu/Projects/badminton-highlight-indexer/.claude/worktrees/tier0-eval

9e03e5a [feat/tier0-rally-seg-eval]

C:/Users/avidu/Projects/badminton-highlight-indexer/.claude/worktrees/tier1-windowing

a3b209a [master]

C:/Users/avidu/Projects/badminton-highlight-indexer/.claude/worktrees/w2-netcross

effd920 [feat/w2-netcross-gate]

C:/Users/avidu/Projects/badminton-highlight-indexer/.claude/worktrees/w2-safeguard

5ba6955 [feat/w2-never-empty-safeguard]

66. assistant (2026-06-18T01:15:45.802Z)

Done. PR [#210](<https://github.com/avidullu/badminton-highlight-indexer/pull/210>) is open.

What I did

Fixed the same latent `KeyError: 'shuttle'` bug in both fusion golden-feature loaders that was just fixed in `ablation.py::load\_videos`:

```
- **[backend/eval/fusion_compare.py](backend/eval/fusion_compare.py)** ? added a local
```

```
`_is_fusion_schema(candidates)` predicate (full-fusion iff every candidate has
`shuttle`+`person`; audio optional) and a module logger; `load_videos` now skips windowing-only
files with an info log instead of crashing.
- [backend/eval/fusion_audio.py](backend/eval/fusion_audio.py) ? imports that predicate from
`fusion_compare` (already a dependency ? `delta_stats` came from there, so no awkward import)
and gates `load_videos_with_audio`. The skip is checked before the audio-presence guard, so
a genuine fusion file that just hasn't been --augment-ed still raises, but windowing-only
files are dropped cleanly.
- One regression test per loader asserting a mixed corpus loads only the fusion files.
```

Key decisions

```
- Duplicated the predicate rather than importing from `ablation.py` ? that fix is on an
unmerged branch (fix/ablation-litmus-mixed-schema`), so importing would stack this PR on it.
The predicate's a one-liner; the docstring records the intentional duplication for a future
consolidation once both land. This honored "do not stack on the ablation fix."
```

Verification (local, in an isolated worktree off `origin/master`)

```
- `python run_all_tests.py --cov=backend --cov=training --cov-fail-under=70` ? 846 passed, 7
skipped, coverage 80.66% (?70 floor, no regression)
- `ruff check .` ? clean
- `mypy backend training` ? clean (no issues in 104 files)
- Both new tests verified to fail without the fix (KeyError / the misleading `SystemExit`)
and pass with it ? they're genuine, not vacuous.
```

Branch was cut from `origin/master`, HEAD re-verified before commit (`origin/master..HEAD` = my single commit only), explicit paths staged, and the worktree removed after pushing ? per the shared-checkout safety protocol.

One thing worth flagging: when `fix/ablation-litmus-mixed-schema`` and this both merge, there'll be two copies of the `\_is\_fusion\_schema` one-liner. A small follow-up could consolidate them into a shared `backend/eval` helper ? but that's intentionally out of scope here to avoid the stacking dependency.

67. user (2026-06-18T05:32:11.470Z)

I think both are merged, can you do the small follow up as a diligence cleanup?

68. assistant (2026-06-18T05:32:31.107Z)

Let me scout the current state first ? confirm both fixes are merged, pull master, and map all copies/usages of `\_is\_fusion\_schema` before deciding the shared location.

69. user (2026-06-18T05:32:37.709Z)

```
=== origin/master tip ===
f5f339a fix(eval): fusion loaders skip windowing-only files on a mixed golden corpus (#210)
db96ddc docs: capture external due-diligence review inputs (research, pick-up-later) (#207)
af5b3e6 docs(rally): tracked doc for closing the detection accuracy gap (golden-driven, per-
video) (#211)
55e4c89 docs(archive): LOVO threshold re-sweep at n=13 ? milestone thresholds hold (#214)
bd33085 fix(scripts): unwrap segmenter Result before len() (motion crashes the default eval
path) (#213)
```

```
=== is ablation fix merged? ===
ablation NOT merged (by contains)
```

```
=== is #210 (fusion loaders) merged? ===
fusion NOT merged (by contains)
```

70. assistant (2026-06-18T05:32:42.831Z)

Squash-merges, so `--contains`` won't match ? #210 is clearly at the tip (f5f339a). Let me verify by content: check whether `\_is\_fusion\_schema` exists in both `ablation.py` and `fusion\_compare.py` on `origin/master`.

71. user (2026-06-18T05:32:45.545Z)

```
=== _is_fusion_schema definitions/uses on origin/master ===
origin/master:backend/eval/ablation.py:386:def _is_fusion_schema(candidates: Sequence[Dict[str,
Any]]) -> bool:
origin/master:backend/eval/ablation.py:406:    ablatable subset cleanly (see
`_is_fusion_schema`)."""
origin/master:backend/eval/ablation.py:412:    if not _is_fusion_schema(cands):
origin/master:backend/eval/fusion_audio.py:23:from backend.eval.fusion_compare import
_is_fusion_schema, delta_stats
origin/master:backend/eval/fusion_audio.py:60:    crashed on, so a mixed corpus loads the fusion
subset cleanly (see `_is_fusion_schema`).
origin/master:backend/eval/fusion_audio.py:68:    if not _is_fusion_schema(cands):
origin/master:backend/eval/fusion_compare.py:38:def _is_fusion_schema(candidates:
Sequence[Dict[str, Any]]) -> bool:
origin/master:backend/eval/fusion_compare.py:61:    fusion subset cleanly (see
`_is_fusion_schema`)."""
origin/master:backend/eval/fusion_compare.py:67:    if not _is_fusion_schema(cands):
origin/master:tests/test_ablation.py:373:def
test_is_fusion_schema_distinguishes_lean_and_full():
origin/master:tests/test_ablation.py:379:    assert ablation._is_fusion_schema(full) is True
origin/master:tests/test_ablation.py:380:    assert ablation._is_fusion_schema(no_audio) is True
origin/master:tests/test_ablation.py:381:    assert ablation._is_fusion_schema(lean) is False
origin/master:tests/test_ablation.py:382:    assert ablation._is_fusion_schema([]) is False
origin/master:tests/test_ablation.py:420:    if ablation._is_fusion_schema(cands):

=== ablation fix merged via PR? search log ===
4bd9d2d fix(eval): ablation litmus tolerates a mixed-schema golden corpus (#209)
```

72. assistant (2026-06-18T05:33:31.536Z)

Both are merged: ablation fix is PR #209 (`\_is\_fusion\_schema` in `ablation.py` + tested in `test\_ablation.py`), fusion fix is #210 (`\_is\_fusion\_schema` in `fusion\_compare.py`, imported by `fusion\_audio.py`). So there are now **two identical definitions**. Let me map the import graph and pick a non-circular shared home.

73. user (2026-06-18T05:33:34.074Z)

```
=== backend/eval modules ===
backend/eval/__init__.py
backend/eval/ablation.py
backend/eval/annotations.py
backend/eval/batch.py
backend/eval/calibrate_local.py
backend/eval/calibrate_wasb.py
backend/eval/calibration.py
backend/eval/classifier.py
backend/eval/distill_local.py
backend/eval/eval_partial_labels.py
backend/eval/eval_stats.py
backend/eval/experiment.py
backend/eval/export_wasb_segments.py
backend/eval/fusion_audio.py
backend/eval/fusion_compare.py
backend/eval/fusion_golden.py
backend/eval/gemini_refine.py
backend/eval/golden_fixtures.py
backend/eval/gt_loader.py
backend/eval/harness.py
backend/eval/metrics.py
backend/eval/per_user_eval.py
backend/eval/per_user_gate.py
backend/eval/per_user_regression.py
backend/eval/promotion.py
backend/eval/rally_seg_eval.py
```

```
backend/eval/rally_seq_proto.py
backend/eval/regression.py
backend/eval/score_calibration.py
backend/eval/segmentation_metrics.py
backend/eval/serve_contrast.py
backend/eval/served_gate_a.py
backend/eval/splits.py
backend/eval/tolerance_metrics.py
backend/eval/training.py
backend/eval/windowing.py
```

```
=== fusion_golden.py imports (potential shared home ? must NOT import
ablation/fusion_compare/fusion_audio) ===
17:from __future__ import annotations
19:import argparse
20:import json
21:import os
22:import time
23:from typing import Any, Dict, List, Optional
25:from backend.pipeline.detectors.tracknet_runner import TrackNetRunner
26:from backend.pipeline.detectors import rally_gate
27:from backend.pipeline.detectors import fusion_features as ff
28:from backend.eval import distill_local
29:from backend.eval.calibrate_wasb import load_golden_gts
30:from backend.eval.training import label_candidates
31:from backend.utils.run_telemetry import RunTelemetry
34:SHUTTLE_FEATURE_NAMES = list(distill_local.FEATURE_NAMES)
35:# The 5 person-cue features (fusion_features.PERSON_FEATURE_NAMES).
36:PERSON_FEATURE_NAMES = list(ff.PERSON_FEATURE_NAMES)
39:def _derive_video_path(spec: Dict[str, Any]) -> str:
50:def extract_video(spec: Dict[str, Any], detector: Optional[Any] = None,
118:         "shuttle": {k: float(v) for k, v in zip(SHUTTLE_FEATURE_NAMES, xs)},
129:def run(manifest_path: str, out_dir: str, frame_skip: int = 3, iou: float = 0.5,
180:def main() -> None:
```

74. assistant (2026-06-18T05:34:04.449Z)

`fusion\_golden.py` is heavy (pulls in `TrackNetRunner`, `rally\_gate`, `fusion\_features`) ? wrong home for a pure dict-key predicate; it'd drag heavy deps into `ablation.py`. Let me check the lighter shared candidate `experiment.py` (already imported by all three consumers) and confirm `ablation.py`'s current imports + no circular risk.

75. user (2026-06-18T05:34:06.850Z)

```
=== ablation.py imports ===
46:from __future__ import annotations
48:import argparse
49:import glob
50:import json
51:import logging
52:import os
53:from dataclasses import dataclass, field
54:from typing import Any, Dict, List, Optional, Sequence, Tuple
56:from backend.eval import experiment
57:from backend.eval.experiment import Variant, VideoCandidates

=== experiment.py imports (is it light? circular?) ===
38:from __future__ import annotations
40:import json
41:import logging
42:import os
43:from dataclasses import dataclass, field
44:from datetime import datetime, timezone
45:from hashlib import sha1
46:from typing import Any, Callable, Dict, List, Optional, Sequence
```



```

48:from backend.eval.calibration import pct_improvement
49:from backend.eval.classifier import LogisticRegression
50:from backend.eval.gemini_refine import merge_overlaps
51:from backend.eval.metrics import Interval, match_intervals, score
52:from backend.eval.regression import gt_hash
53:from backend.eval.training import label_candidates
54:from backend.eval import eval_stats as _stats          # C1: CIs + paired sign test
55:from backend.eval import segmentation_metrics as _segm  # C4: F1@k + segment-count ratio
56:from backend.eval.score_calibration import fit_isotonic, fit_platt  # C2: calibrate cue
scores

```

76. assistant (2026-06-18T05:35:27.219Z)

All three consumers (`ablation`, `fusion\_compare`, `fusion\_audio`) already import `experiment`, and `experiment.py` imports none of them ? so it's a ****circular-safe, zero-new-edge**** home, and it owns `VideoCandidates` (the type these JSONs deserialize into). That's the right shared spot. Let me look at where to place it and how the existing predicate test is structured.

77. user (2026-06-18T05:35:30.239Z)

```

=== test_experiment.py exists? ===
tests/test_experiment.py

=== experiment.py def inventory + does it touch *_golden_features.json / load_video_candidates?
===
10: - `backend.eval.training`      ? `label_candidates` (y by the same IoU matcher)
35: `load_video_candidates`, which reads persisted candidates via
53: from backend.eval.training import label_candidates
67: class ExperimentError(ValueError):
77: class Variant:
107:     name: str
156: #: The pinned, frozen baseline: candidates as detected, merged, no gate, no learned
161: # ----- persisted candidates
165: class VideoCandidates:
171:     with `load_video_candidates`, or directly in tests.
174:     name: str
180: def _feature_column(vc: VideoCandidates, name: str) -> List[float]:
191:     f"feature {name!r} has {len(col)} values but {vc.name} has {n} candidates"
195: def _feature_matrix(vc: VideoCandidates, feature_names: Sequence[str]) -> List[List[float]]:
200: def _gate_mask(variant: Variant, vc: VideoCandidates) -> List[bool]:
213: def predict(variant: Variant, vc: VideoCandidates,
243: def _calibrate_and_tune(variant: Variant, model: LogisticRegression,
257:     y.extend(label_candidates(vc.intervals, vc.gts or [], iou))
281: def _train(variant: Variant, videos: Sequence[VideoCandidates], iou: float) ->
LogisticRegression:
282:     """Fit the variant's classifier on the pooled candidates of ``videos`` (labels via the
290:     y.extend(label_candidates(vc.intervals, vc.gts, iou))
292:     raise ExperimentError("cannot train a learned variant on zero candidates")
296: def evaluate_variant(videos: Sequence[VideoCandidates], variant: Variant,
359: def _ci_block(eval_v: Dict[str, Any]) -> Dict[str, Any]:
364: def _segmentation_block(videos: Sequence[VideoCandidates], eval_v: Dict[str, Any]) ->
Dict[str, Any]:
384: def honest_summary(videos: Sequence[VideoCandidates], eval_a: Dict[str, Any],
403: def compare(videos: Sequence[VideoCandidates], variant_a: Variant, variant_b: Variant,
454: def compare_unlabeled(video: VideoCandidates, variant_a: Variant, variant_b: Variant,
504: def record_experiment(result: Dict[str, Any], path: str = DEFAULT_LEADERBOARD_PATH,
543: def load_video_candidates(db: Any, video_id: str, *, detector: Optional[str] = None,

```

78. assistant (2026-06-18T05:35:41.250Z)

`experiment.py` owns `VideoCandidates` (line 165) and `load\_video\_candidates` ? the predicate belongs right after the dataclass. `tests/test\_experiment.py` exists, so the canonical unit test can live there honestly. Let me set up an isolated worktree off `origin/master` and read the live files.

79. user (2026-06-18T05:35:44.078Z)

Preparing worktree (new branch 'chore/consolidate-fusion-schema-predicate')
branch 'chore/consolidate-fusion-schema-predicate' set up to track 'origin/master'.
HEAD is now at f5f339a fix(eval): fusion loaders skip windowing-only files on a mixed golden corpus (#210)
=== HEAD ===
f5f339a fix(eval): fusion loaders skip windowing-only files on a mixed golden corpus (#210)
chore/consolidate-fusion-schema-predicate

80. user (2026-06-18T05:35:48.628Z)

```
383      # ----- corpus
384      I/O
385
386      def _is_fusion_schema(candidates: Sequence[Dict[str, Any]]) -> bool:
387          """True iff every candidate row carries the per-cue feature dicts the ablation
388          scorer reads
389              (`shuttle` + `person`; `audio` stays optional ? older JSONs predate it).
390              The velocity-windowing / distill path persists a LEANER `*_golden_features.json`
391              row
392              (`start`/`end` + velocity stats, no cue dicts) for clips that never went through
393              fusion
394              feature extraction. `load_videos` SKIPS those rather than crash on
395              `c["shuttle"]` ? so a
396              mixed corpus (some fusion files, some windowing-only) loads the ablatable subset
397              cleanly.
398              An empty candidate list is not ablatable either."""
399              return bool(candidates) and all("shuttle" in c and "person" in c for c in
400              candidates)
401
402      def load_videos(features_dir: str) -> List[VideoCandidates]:
403          """Load every FULL-FUSION-SCHEMA `*_golden_features.json` into a
404          `VideoCandidates`
405          (intervals + shuttle + person + audio columns + golden gts). Audio columns are
406          optional
407          (older JSONs predate the audio augment) ? a missing block defaults those columns to
408          0.0 so
409          the loader never crashes, mirroring `experiment._feature_column`'s tolerance.
410
411          Leaner velocity-windowing-only files (no `shuttle`/`person` cue dicts ? the
412          schema the
413          R3/R4 corpus-growth path persists) are SKIPPED, not crashed on, so a mixed corpus
414          loads the
415          ablatable subset cleanly (see `_is_fusion_schema`)."""
416          videos: List[VideoCandidates] = []
417          for path in sorted(glob.glob(os.path.join(features_dir, "*_golden_features.json"))):
418              with open(path, encoding="utf-8") as f:
419                  d = json.load(f)
420                  cands = d["candidates"]
421                  if not _is_fusion_schema(cands):
422                      logger.info("ablation.load_videos: skipping %s ? windowing-only schema "
423                                  "(no shuttle/person cue dicts), not an ablatable fusion-feature
424                                  file",
425                                  os.path.basename(path))
426                      continue
427                  intervals = [(float(c["start"]), float(c["end"])) for c in cands]
428                  features: Dict[str, List[float]] = {}
429                  for fn in SHUTTLE_COLUMNS:
430                      features[fn] = [float(c["shuttle"][fn]) for c in cands]
431                  for fn in PERSON_COLUMNS:
432                      features[fn] = [float(c["person"][fn]) for c in cands]
```

```

423         for fn in AUDIO_COLUMNS:
424             features[fn] = [float((c.get("audio") or {}).get(fn, 0.0)) for c in cands]

```

81. assistant (2026-06-18T05:35:47.892Z)

Now let me read the live files in the worktree ? the current `\_is\_fusion\_schema` defs in `ablation.py` and `fusion\_compare.py`, the `experiment.py` placement site, and the existing predicate test.

82. user (2026-06-18T05:35:52.205Z)

```

368         "candidates": cands, "gts": [[0.0, 5.0]])
369     with open(os.path.join(d, f"{name}_golden_features.json"), "w", encoding="utf-8") as
f:
370         json.dump(data, f)
371
372
373     def test_is_fusion_schema_distinguishes_lean_and_full():
374         """The schema predicate keeps full-fusion rows (shuttle+person, audio optional) and
rejects
375         the leaner velocity-windowing rows (no cue dicts); an empty candidate list is not
ablatable."""
376         full = [{"start": 0.0, "end": 1.0, "shuttle": {}, "person": {}, "audio": {}}]
377         no_audio = [{"start": 0.0, "end": 1.0, "shuttle": {}, "person": {}}] #
audio optional
378         lean = [{"start": 0.0, "end": 1.0, "mean_velocity": 1.0, "track_id_count": 1}]
379         assert ablation._is_fusion_schema(full) is True
380         assert ablation._is_fusion_schema(no_audio) is True
381         assert ablation._is_fusion_schema(lean) is False
382         assert ablation._is_fusion_schema([]) is False
383
384
385     def test_load_videos_skips_windowing_only_schema(tmp_path):
386         """Mixed-corpus regression (the n=11 `KeyError: 'shuttle'`): full-fusion files
load; the
387         leaner velocity-windowing files (the 5 newer real clips) are SKIPPED rather than
crashed on,
388         and the litmus path runs end-to-end over the filtered ablatable subset."""
389         d = str(tmp_path)
390         _write_learned_corpus(d, n=6) # 6 full-fusion files
(v0..v5)
391         for name in ("mahadevpura_1", "GX010128", "Badminton_BXH_2"): # 3 leaner real-clip
names
392             _write_windowing_only(d, name)
393             # only the fusion-schema files load ? the leaner ones never crash the loader
394             videos = ablation.load_videos(d)
395             assert {v.name for v in videos} == {f"v{k}" for k in range(6)}
396             # ?and the litmus path (which reads the cue columns) runs cleanly on the mixed
corpus
397             report = run_ablation(videos, ablation.litmus_config())
398             assert [r.signal for r in report.rows] == ["gate_a"]
399             assert report.num_videos == 6
400
401
402     # ----- the
golden litmus
403
404     GOLDEN_DIR = os.path.join(
405         os.path.dirname(os.path.abspath(__file__)), "output")
406
407
408     def _golden_present() -> bool:
409         """True iff ?6 FULL-FUSION-SCHEMA golden files exist (the litmus needs the
shuttle/person cue
410         columns). Counts only ablatable files ? leaner velocity-windowing files are skipped

```

```

by
411     ``load_videos``, so counting them would trip the litmus on an un-ablatable corpus
(the n=11
412     mixed-schema bug)."""
413     n = 0
414     for p in glob.glob(os.path.join(GOLDEN_DIR, "*_golden_features.json")):
415         try:
416             with open(p, encoding="utf-8") as f:
417                 cands = json.load(f).get("candidates") or []
418             except (OSError, ValueError):
419                 continue
420             if ablation._is_fusion_schema(cands):
421                 n += 1
422     return n >= 6
423
424
425     @pytest.mark.skipif(not _golden_present(),
426                         reason="golden output/*_golden_features.json absent (run
fusion_golden first)")
427     def test_litmus_reproduces_gate_a():

```

83. assistant (2026-06-18T05:36:26.412Z)

Now I have the full picture. Two `test\_ablation.py` references: the canonical unit test (lines 373-382) and `\_golden\_present()` (line 420). Let me check `Sequence` usage (to clean up imports after removing the defs), confirm no other references exist, and inspect `test\_experiment.py`'s structure.

84. user (2026-06-18T05:36:31.093Z)

```

=== Sequence usage in ablation.py (besides the def being removed) ===
54:from typing import Any, Dict, List, Optional, Sequence, Tuple
211:def _ablated_variant(full: Variant, columns: Sequence[str], gate_knob: Optional[str]) ->
Variant:
248:def ablate_one(videos: Sequence[VideoCandidates], cfg: AblationConfig,
298:def _collapse_to_groups(signals: Sequence[Signal]) -> List[Signal]:
318:def run_ablation(videos: Sequence[VideoCandidates], cfg: AblationConfig) -> AblationReport:
386:def _is_fusion_schema(candidates: Sequence[Dict[str, Any]]) -> bool:
453:     def _fmt_row(cells: Sequence[str]) -> str:
=== Sequence usage in fusion_compare.py ===
19:from typing import Any, Dict, List, Sequence
38:def _is_fusion_schema(candidates: Sequence[Dict[str, Any]]) -> bool:
=== ALL _is_fusion_schema references repo-wide (sanity) ===
backend/eval/ablation.py:386:def _is_fusion_schema(candidates: Sequence[Dict[str, Any]]) ->
bool:
backend/eval/ablation.py:406:     ablatable subset cleanly (see ``_is_fusion_schema``)."""
backend/eval/ablation.py:412:         if not _is_fusion_schema(cands):
backend/eval/fusion_audio.py:23:from backend.eval.fusion_compare import _is_fusion_schema,
delta_stats
backend/eval/fusion_audio.py:60:     crashed on, so a mixed corpus loads the fusion subset
cleanly (see ``_is_fusion_schema``).
backend/eval/fusion_audio.py:68:         if not _is_fusion_schema(cands):
backend/eval/fusion_compare.py:38:def _is_fusion_schema(candidates: Sequence[Dict[str, Any]]) ->
bool:
backend/eval/fusion_compare.py:61:     fusion subset cleanly (see ``_is_fusion_schema``)."""
backend/eval/fusion_compare.py:67:         if not _is_fusion_schema(cands):
tests/test_ablation.py:373:def test_is_fusion_schema_distinguishes_lean_and_full():
tests/test_ablation.py:379:     assert ablation._is_fusion_schema(full) is True
tests/test_ablation.py:380:     assert ablation._is_fusion_schema(no_audio) is True
tests/test_ablation.py:381:     assert ablation._is_fusion_schema(lean) is False
tests/test_ablation.py:382:     assert ablation._is_fusion_schema([]) is False
tests/test_ablation.py:420:         if ablation._is_fusion_schema(cands):
=== test_experiment.py imports/head ===
"""Tests for the experiment harness (backend.eval.experiment).

```

Pure/offline ? synthetic candidate rows, no GPU/WSL. The load-bearing test is the ****no-regression invariant**** (``test_disabled_cues_byte_identical_to_control``): a variant that merely carries a disabled cue must predict exactly what control predicts. That is the guarantee that lets new cues be merged freely (``EXPERIMENT_HARNESS.md` §6`).

```
import json

import pytest

from backend.eval.classifier import LogisticRegression
from backend.eval.experiment import (
    CONTROL,
    ExperimentError,
    Variant,
    VideoCandidates,
    compare,
    compare_unlabeled,
    evaluate_variant,
    load_video_candidates,
    predict,
    record_experiment,
)
from backend.infrastructure.database import Database
Shell cwd was reset to C:\Users\avidu\Projects\badminton-highlight-indexer
```

85. assistant (2026-06-18T05:36:44.296Z)

``Sequence`` stays in ``ablation.py`` (used in 5 other signatures) but must be ****removed**** from ``fusion_compare.py`` (only the predicate used it). All references are contained. Let me read the exact ``VideoCandidates`` block in ``experiment.py`` to place the canonical predicate.

86. user (2026-06-18T05:36:45.425Z)

```
161 # ----- persisted candidates
162
163
164 @dataclass
165 class VideoCandidates:
166     """One video's persisted detection, ready for offline re-scoring.
167
168     ``intervals`` are the candidate windows; ``features`` is column-major
169     (``feature_name -> per-candidate value``, each list aligned to ``intervals``);
170     ``gts`` are golden labels (None for new/unlabeled footage). Build one from the DB
171     with ``load_video_candidates``, or directly in tests.
172     """
173
174     name: str
175     intervals: List[Interval]
176     features: Dict[str, List[float]] = field(default_factory=dict)
177     gts: Optional[List[Interval]] = None
178
179
180 def _feature_column(vc: VideoCandidates, name: str) -> List[float]:
181     """Persisted feature column, defaulting missing/short columns to 0.0 (matches
182     `training.extract_yolo_features`, which lets a sparse/old row still vectorise)."""
183     col = vc.features.get(name)
184     n = len(vc.intervals)
185     if col is None:
186         logger.warning("variant references feature %r absent from %s ? treating as 0.0",
187                        name, vc.name)
188         return [0.0] * n
189     if len(col) != n:
190         raise ExperimentError(
191             f"feature {name!r} has {len(col)} values but {vc.name} has {n} candidates")
192     return [float(x) for x in col]
```

```

193
194
195     def _feature_matrix(vc: VideoCandidates, feature_names: Sequence[str]) ->
List[List[float]]:
196         cols = [_feature_column(vc, name) for name in feature_names]

```

87. assistant (2026-06-18T05:36:58.929Z)

The placement is clear ? right after the `VideoCandidates` dataclass. Let me make all the edits. First, add the canonical predicate to `experiment.py`:

88. user (2026-06-18T05:37:04.581Z)

The file C:\Users\avidu\Projects\bhi-consolidate-schema\backend\eval\experiment.py has been updated successfully. (file state is current in your context ? no need to Read it back)

89. assistant (2026-06-18T05:37:10.918Z)

Now `ablation.py` ? import the shared predicate and remove its local copy:

90. user (2026-06-18T05:37:12.513Z)

The file C:\Users\avidu\Projects\bhi-consolidate-schema\backend\eval\ablation.py has been updated successfully. (file state is current in your context ? no need to Read it back)

91. user (2026-06-18T05:37:20.124Z)

The file C:\Users\avidu\Projects\bhi-consolidate-schema\backend\eval\ablation.py has been updated successfully. (file state is current in your context ? no need to Read it back)

92. assistant (2026-06-18T05:37:29.361Z)

Now `fusion\_compare.py` ? drop `Sequence`, import the shared predicate, remove the local def. Let me read its current header:

93. user (2026-06-18T05:37:29.943Z)

```

12     from __future__ import annotations
13
14     import argparse
15     import glob
16     import json
17     import logging
18     import os
19     from typing import Any, Dict, List, Sequence
20
21     from backend.eval import experiment
22     from backend.eval.eval_stats import paired_delta_ci
23     from backend.eval.fusion_golden import PERSON_FEATURE_NAMES, SHUTTLE_FEATURE_NAMES
24
25     logger = logging.getLogger(__name__)
26
27
28     def delta_stats(res: Dict[str, Any], metric: str = "f1") -> Dict[str, Any]:
29         """Paired (by video) B?A summary for one metric: mean delta + bootstrap CI + sign
test
30         (eval_stats, C1 ? "never a bare mean"). Aligns the two variants' per-video held-out
scores
31         by video name, so the headline ?F1 comes with how-wide + did-it-win-on-enough-
videos."""
32         by_a = {p["video"]: p[metric] for p in res["variant_a"]["per_video"]}
33         by_b = {p["video"]: p[metric] for p in res["variant_b"]["per_video"]}
34         vids = [p["video"] for p in res["variant_a"]["per_video"]]
35         return paired_delta_ci([by_a[v] for v in vids], [by_b[v] for v in vids])

```

```

36
37
38     def _is_fusion_schema(candidates: Sequence[Dict[str, Any]]) -> bool:
39         """True iff every candidate row carries the per-cue feature dicts the fusion loaders
read
40             (`shuttle` + `person`; `audio` stays optional ? it is added later by
41             `fusion_audio.augment` and older JSONs predate it).
42
43             The velocity-windowing / distill corpus-growth path persists a LEANER
44             ``*_golden_features.json`` row (`start`/`end` + velocity stats, no cue dicts)
for clips
45             that never went through fusion feature extraction. The loaders SKIP those rather
than crash
46             on ``c["shuttle"]`` ? so a MIXED corpus (some fusion files, some windowing-only)
loads the
47             fusion subset cleanly. An empty candidate list is not a fusion file either.
48
49             (Kept local to the fusion loaders; `ablation.py` carries its own copy of this one-
liner ?
50             the two modules are independent and the shared form would force an awkward
import.)"""
51         return bool(candidates) and all("shuttle" in c and "person" in c for c in
candidates)

```

94. user (2026-06-18T05:37:35.208Z)

The file C:\Users\avidu\Projects\bhi-consolidate-schema\backend\eval\fusion_compare.py has been updated successfully. (file state is current in your context ? no need to Read it back)

95. user (2026-06-18T05:37:44.062Z)

The file C:\Users\avidu\Projects\bhi-consolidate-schema\backend\eval\fusion_compare.py has been updated successfully. (file state is current in your context ? no need to Read it back)

96. assistant (2026-06-18T05:37:49.460Z)

Now `fusion\_audio.py` ? repoint the predicate import from `fusion\_compare` to the canonical `experiment`:

97. user (2026-06-18T05:37:50.282Z)

```

14
15     import argparse
16     import glob
17     import json
18     import logging
19     import os
20     from typing import Any, Dict, List
21
22     from backend.eval import experiment
23     from backend.eval.fusion_compare import _is_fusion_schema, delta_stats
24     from backend.eval.fusion_golden import PERSON_FEATURE_NAMES, SHUTTLE_FEATURE_NAMES,
derive_video_path
25     from backend.pipeline.detectors import audio_features as af
26     from backend.pipeline.audio.hit_detector import detect_hits_in_video
27
28     logger = logging.getLogger(__name__)
29

```

98. user (2026-06-18T05:37:58.189Z)

The file C:\Users\avidu\Projects\bhi-consolidate-schema\backend\eval\fusion_audio.py has been updated successfully. (file state is current in your context ? no need to Read it back)

99. assistant (2026-06-18T05:38:29.022Z)

Now the tests. Let me move the canonical predicate unit test to `test\_experiment.py` (its true home now). First read `test\_experiment.py` structure:

100. user (2026-06-18T05:38:29.547Z)

```

11
12     from backend.eval.classifier import LogisticRegression
13     from backend.eval.experiment import (
14         CONTROL,
15         ExperimentError,
16         Variant,
17         VideoCandidates,
18         compare,
19         compare_unlabeled,
20         evaluate_variant,
21         load_video_candidates,
22         predict,
23         record_experiment,
24     )
25     from backend.infrastructure.database import Database
26
27
28     # A video whose third candidate is a held-shuttle false fire (net_crossings=0): it does
29     # not overlap any golden rally, so it is a negative by construction.
30     def _held_shuttle_video(name="v", gts=None):
31         return VideoCandidates(
32             name=name,
33             intervals=[(0.0, 10.0), (20.0, 30.0), (12.0, 13.0)],
34             features={"net_crossings": [4.0, 3.0, 0.0]},
35             gts=gts if gts is not None else [(0.0, 10.0), (20.0, 30.0)],
36         )
37
38
39     # Three videos where feature "x" perfectly separates rallies (x=5) from junk (x=0),
40     # so an honest LOVO learned variant should recover F1?1.0 on each held-out video.
41     def _separable_videos():
42         vids = []
43         for k in range(3):
44             base = 100.0 * k
45             vids.append(VideoCandidates(
46                 name=f"sep{k}",
47                 intervals=[(base + 0, base + 10), (base + 20, base + 30),
48                     (base + 40, base + 41), (base + 50, base + 51)],
49                 features={"x": [5.0, 5.0, 0.0, 0.0]},
50                 gts=[(base + 0, base + 10), (base + 20, base + 30)],
51             ))
52         return vids
53
54
55     # ----- Variant
56
57     def test_variant_roundtrip_and_hash_stability():
58         v = Variant(name="gateA", min_crossings=2, cue_flags={"person": False})
59         assert Variant.from_dict(v.to_dict()) == v
60         assert v.spec_hash() == v.spec_hash() # stable
61         assert CONTROL.spec_hash() == Variant(name="control").spec_hash()
62         assert v.spec_hash() != CONTROL.spec_hash() # different recipe ?
63         different hash
64
65     def test_from_dict_ignores_unknown_keys():

```

101. user (2026-06-18T05:38:46.416Z)

The file C:\Users\avidu\Projects\bhi-consolidate-schema\tests\test_experiment.py has been updated successfully. (file state is current in your context ? no need to Read it back)

102. user (2026-06-18T05:38:54.117Z)

The file C:\Users\avidu\Projects\bhi-consolidate-schema\tests\test_experiment.py has been updated successfully. (file state is current in your context ? no need to Read it back)

103. assistant (2026-06-18T05:38:56.625Z)

Now remove the now-duplicated unit test from `test\_ablation.py`:

104. user (2026-06-18T05:39:00.610Z)

The file C:\Users\avidu\Projects\bhi-consolidate-schema\tests\test_ablation.py has been updated successfully. (file state is current in your context ? no need to Read it back)

105. assistant (2026-06-18T05:39:14.287Z)

Now let me verify `test\_ablation.py`'s remaining `ablation.\_is\_fusion\_schema` reference still resolves (via re-import) and run the targeted test files:

106. user (2026-06-18T05:39:40.534Z)

```
=== test_ablation imports ablation as module? ===
25:from backend.eval import ablation
=== run targeted tests ===
.....SS..... [100%]
60 passed, 2 skipped in 23.07s
Shell cwd was reset to C:\Users\avidu\Projects\badminton-highlight-indexer
```

107. assistant (2026-06-18T05:39:44.657Z)

All 60 pass (2 skipped = golden-litmus skipif). Now the full verification gate ? suite with coverage, ruff, mypy:

108. user (2026-06-18T05:41:29.868Z)

```
=== FULL SUITE + COVERAGE ===
TOTAL                        8009   1546   81%
Required test coverage of 70% reached. Total coverage: 80.70%
855 passed, 7 skipped, 7 warnings in 100.00s (0:01:39)
=====
RESULT: PASS - all tests green
=====
Shell cwd was reset to C:\Users\avidu\Projects\badminton-highlight-indexer
```

109. assistant (2026-06-18T05:41:36.029Z)

Suite green: **855 passed, 7 skipped**, coverage **80.70%**. Now ruff + mypy:

110. user (2026-06-18T05:42:40.359Z)

```
=== RUFF ===
All checks passed!
=== MYPY ===
backend\annotations\base.py:85: note: By default the bodies of untyped functions are not
checked, consider using --check-untyped-defs [annotation-unchecked]
backend\sports\__init__.py:7: note: By default the bodies of untyped functions are not checked,
consider using --check-untyped-defs [annotation-unchecked]
backend\pipeline\validators\base.py:36: note: By default the bodies of untyped functions are not
checked, consider using --check-untyped-defs [annotation-unchecked]
backend\providers\__init__.py:7: note: By default the bodies of untyped functions are not
```

```
checked, consider using --check-untyped-defs [annotation-unchecked]
Success: no issues found in 104 source files
Shell cwd was reset to C:\Users\avidu\Projects\badminton-highlight-indexer
```

111. assistant (2026-06-18T05:42:55.645Z)

All gates green. Let me review the full diff and do a final repo-wide sweep for any stale reference to the old import location.

112. user (2026-06-18T05:42:59.806Z)

```
=== STALE REFERENCE SWEEP: any import of _is_fusion_schema from fusion_compare, or defs outside
experiment.py? ===
backend/eval/experiment.py:180:def _is_fusion_schema(candidates: Sequence[Dict[str, Any]]) ->
bool:
```

```
=== all _is_fusion_schema references now ===
backend/eval/ablation.py:57:from backend.eval.experiment import Variant, VideoCandidates,
_is_fusion_schema
backend/eval/ablation.py:394:    ablatable subset cleanly (see ``_is_fusion_schema``)."""
backend/eval/ablation.py:400:        if not _is_fusion_schema(cands):
backend/eval/experiment.py:180:def _is_fusion_schema(candidates: Sequence[Dict[str, Any]]) ->
bool:
backend/eval/fusion_audio.py:23:from backend.eval.experiment import _is_fusion_schema
backend/eval/fusion_audio.py:61:    crashed on, so a mixed corpus loads the fusion subset
cleanly (see ``_is_fusion_schema``).
backend/eval/fusion_audio.py:69:        if not _is_fusion_schema(cands):
backend/eval/fusion_compare.py:23:from backend.eval.experiment import _is_fusion_schema
backend/eval/fusion_compare.py:46:    fusion subset cleanly (see ``_is_fusion_schema``)."""
backend/eval/fusion_compare.py:52:        if not _is_fusion_schema(cands):
Binary file backend/eval/__pycache__/ablation.cpython-313.pyc matches
Binary file backend/eval/__pycache__/experiment.cpython-313.pyc matches
Binary file backend/eval/__pycache__/fusion_audio.cpython-313.pyc matches
Binary file backend/eval/__pycache__/fusion_compare.cpython-313.pyc matches
tests/test_ablation.py:408:        if ablation._is_fusion_schema(cands):
tests/test_experiment.py:18:    _is_fusion_schema,
tests/test_experiment.py:58:def test_is_fusion_schema_distinguishes_lean_and_full():
tests/test_experiment.py:66:    assert _is_fusion_schema(full) is True
tests/test_experiment.py:67:    assert _is_fusion_schema(no_audio) is True
tests/test_experiment.py:68:    assert _is_fusion_schema(lean) is False
tests/test_experiment.py:69:    assert _is_fusion_schema([]) is False
Binary file tests/__pycache__/test_ablation.cpython-313-pytest-9.0.3.pyc matches
Binary file tests/__pycache__/test_experiment.cpython-313-pytest-9.0.3.pyc matches
```

```
=== DIFF STAT ===
backend/eval/ablation.py      | 14 +-----
backend/eval/experiment.py    | 18 ++++++
backend/eval/fusion_audio.py  | 3 +-
backend/eval/fusion_compare.py| 19 ++-----
tests/test_ablation.py        | 12 -----
tests/test_experiment.py      | 17 ++++++
6 files changed, 40 insertions(+), 43 deletions(-)
Shell cwd was reset to C:\Users\avidu\Projects\badminton-highlight-indexer
```

113. assistant (2026-06-18T05:43:06.127Z)

Clean: single definition in `experiment.py:180`, all three consumers import it, no stale `fusion\_compare` imports. The `.pyc` matches are gitignored `\_\_pycache\_\_`. Let me review the full diff content:

114. user (2026-06-18T05:43:07.618Z)

```
diff --git a/backend/eval/ablation.py b/backend/eval/ablation.py
index 968eefd..8324eb2 100644
```

```

--- a/backend/eval/ablation.py
+++ b/backend/eval/ablation.py
@@ -54,7 +54,7 @@ from dataclasses import dataclass, field
    from typing import Any, Dict, List, Optional, Sequence, Tuple

    from backend.eval import experiment
- from backend.eval.experiment import Variant, VideoCandidates
+ from backend.eval.experiment import Variant, VideoCandidates, _is_fusion_schema

    logger = logging.getLogger(__name__)

@@ -383,18 +383,6 @@ def litmus_config() -> AblationConfig:
    # ----- corpus I/O

- def _is_fusion_schema(candidates: Sequence[Dict[str, Any]]) -> bool:
-     """True iff every candidate row carries the per-cue feature dicts the ablation scorer reads
-     (``shuttle`` + ``person``; ``audio`` stays optional ? older JSONs predate it).
-
-     The velocity-windowing / distill path persists a LEANER ``*_golden_features.json`` row
-     (``start``/``end`` + velocity stats, no cue dicts) for clips that never went through fusion
-     feature extraction. ``load_videos`` SKIPS those rather than crash on ``c["shuttle"]`` ? so
a
-     mixed corpus (some fusion files, some windowing-only) loads the ablatable subset cleanly.
-     An empty candidate list is not ablatable either."""
-     return bool(candidates) and all("shuttle" in c and "person" in c for c in candidates)
-
-
    def load_videos(features_dir: str) -> List[VideoCandidates]:
        """Load every FULL-FUSION-SCHEMA ``*_golden_features.json`` into a ``VideoCandidates``
        (intervals + shuttle + person + audio columns + golden gts). Audio columns are optional
diff --git a/backend/eval/experiment.py b/backend/eval/experiment.py
index 6b92f3c..67aae66 100644
--- a/backend/eval/experiment.py
+++ b/backend/eval/experiment.py
@@ -177,6 +177,24 @@ class VideoCandidates:
    gts: Optional[List[Interval]] = None

+ def _is_fusion_schema(candidates: Sequence[Dict[str, Any]]) -> bool:
+     """True iff every candidate row carries the per-cue feature dicts the golden-feature
+     loaders
+     read (``shuttle`` + ``person``; ``audio`` stays optional ? it is added later by
+     ``fusion_audio.augment`` and older JSONs predate it).
+
+     The velocity-windowing / distill corpus-growth path persists a LEANER
+     ``*_golden_features.json`` row (``start``/``end`` + velocity stats, no cue dicts) for clips
+     that never went through fusion feature extraction. The loaders that build cue-rich
+     ``VideoCandidates`` from these files ? ``ablation.load_videos``,
+     ``fusion_compare.load_videos``
+     and ``fusion_audio.load_videos_with_audio`` ? SKIP those rather than crash on
+     ``c["shuttle"]``,
+     so a MIXED corpus (some fusion files, some windowing-only) loads the fusion subset cleanly.
+     An empty candidate list is not a fusion file either.
+
+     Single source of truth for the three loaders (consolidated 2026-06-18 after PRs #209/#210
+     each landed an independent copy)."""
+     return bool(candidates) and all("shuttle" in c and "person" in c for c in candidates)
+
+
    def _feature_column(vc: VideoCandidates, name: str) -> List[float]:
        """Persisted feature column, defaulting missing/short columns to 0.0 (matches
        ``training.extract_yolo_features``, which lets a sparse/old row still vectorise)."""
diff --git a/backend/eval/fusion_audio.py b/backend/eval/fusion_audio.py
index d59dd46..295591f 100644

```

```

--- a/backend/eval/fusion_audio.py
+++ b/backend/eval/fusion_audio.py
@@ -20,7 +20,8 @@ import os
    from typing import Any, Dict, List

    from backend.eval import experiment
-from backend.eval.fusion_compare import _is_fusion_schema, delta_stats
+from backend.eval.experiment import _is_fusion_schema
+from backend.eval.fusion_compare import delta_stats
    from backend.eval.fusion_golden import PERSON_FEATURE_NAMES, SHUTTLE_FEATURE_NAMES,
    _derive_video_path
    from backend.pipeline.detectors import audio_features as af
    from backend.pipeline.audio.hit_detector import detect_hits_in_video
diff --git a/backend/eval/fusion_compare.py b/backend/eval/fusion_compare.py
index 9372ba0..2cb13a2 100644
--- a/backend/eval/fusion_compare.py
+++ b/backend/eval/fusion_compare.py
@@ -16,10 +16,11 @@ import glob
    import json
    import logging
    import os
-from typing import Any, Dict, List, Sequence
+from typing import Any, Dict, List

    from backend.eval import experiment
    from backend.eval.eval_stats import paired_delta_ci
+from backend.eval.experiment import _is_fusion_schema
    from backend.eval.fusion_golden import PERSON_FEATURE_NAMES, SHUTTLE_FEATURE_NAMES

    logger = logging.getLogger(__name__)
@@ -35,22 +36,6 @@ def delta_stats(res: Dict[str, Any], metric: str = "f1") -> Dict[str, Any]:
    return paired_delta_ci([by_a[v] for v in vids], [by_b[v] for v in vids])

-def _is_fusion_schema(candidates: Sequence[Dict[str, Any]]) -> bool:
-    """True iff every candidate row carries the per-cue feature dicts the fusion loaders read
-    (`shuttle` + `person`; `audio` stays optional ? it is added later by
-    `fusion_audio.augment` and older JSONs predate it).
-
-    The velocity-windowing / distill corpus-growth path persists a LEANER
-    ``*_golden_features.json`` row (`start`/`end` + velocity stats, no cue dicts) for clips
-    that never went through fusion feature extraction. The loaders SKIP those rather than crash
-    on ``c["shuttle"]`` ? so a MIXED corpus (some fusion files, some windowing-only) loads the
-    fusion subset cleanly. An empty candidate list is not a fusion file either.
-
-    (Kept local to the fusion loaders; `ablation.py` carries its own copy of this one-liner ?
-    the two modules are independent and the shared form would force an awkward import.)"""
-    return bool(candidates) and all("shuttle" in c and "person" in c for c in candidates)
-
-def load_videos(features_dir: str) -> List[experiment.VideoCandidates]:
-    """Load every FULL-FUSION-SCHEMA ``*_golden_features.json`` into a VideoCandidates
-    (intervals + the 10 feature columns + golden gts). The harness derives labels from gts
diff --git a/tests/test_ablation.py b/tests/test_ablation.py
index 224d497..184b074 100644
--- a/tests/test_ablation.py
+++ b/tests/test_ablation.py
@@ -370,18 +370,6 @@ def _write_windowing_only(d: str, name: str) -> None:
    json.dump(data, f)

-def test_is_fusion_schema_distinguishes_lean_and_full():
-    """The schema predicate keeps full-fusion rows (shuttle+person, audio optional) and rejects
-    the leaner velocity-windowing rows (no cue dicts); an empty candidate list is not
-    ablatable."""

```


Refactor complete and fully verified. Now commit ? re-verifying HEAD and staging explicit paths (shared checkout):

118. user (2026-06-18T05:43:43.650Z)

```
=== branch ===
chore/consolidate-fusion-schema-predicate
=== HEAD (should be origin/master tip) ===
f5f339a fix(eval): fusion loaders skip windowing-only files on a mixed golden corpus (#210)
=== commits vs origin/master (empty pre-commit) ===
=== status (only my 6 files) ===
  M backend/eval/ablation.py
  M backend/eval/experiment.py
  M backend/eval/fusion_audio.py
  M backend/eval/fusion_compare.py
  M tests/test_ablation.py
  M tests/test_experiment.py
Shell cwd was reset to C:\Users\avidu\Projects\badminton-highlight-indexer
```

119. assistant (2026-06-18T05:43:54.386Z)

Clean ? HEAD at `origin/master` tip, no sibling commits, only my 6 files. Staging explicit paths and committing:

120. user (2026-06-18T05:44:03.374Z)

```
e268972 refactor(eval): single source of truth for the _is_fusion_schema predicate
Shell cwd was reset to C:\Users\avidu\Projects\badminton-highlight-indexer
```

121. user (2026-06-18T05:44:10.312Z)

```
=== only my commit vs origin/master ===
e268972 refactor(eval): single source of truth for the _is_fusion_schema predicate
=== LF clean? ===
0 CR ? LF clean
=== push ===
remote:
branch 'chore/consolidate-fusion-schema-predicate' set up to track 'origin/chore/consolidate-
fusion-schema-predicate'.
To https://github.com/avidullu/badminton-highlight-indexer.git
 * [new branch]      chore/consolidate-fusion-schema-predicate -> chore/consolidate-fusion-
schema-predicate
Shell cwd was reset to C:\Users\avidu\Projects\badminton-highlight-indexer
```

122. user (2026-06-18T05:44:27.031Z)

```
https://github.com/avidullu/badminton-highlight-indexer/pull/215
Shell cwd was reset to C:\Users\avidu\Projects\badminton-highlight-indexer
```

123. assistant (2026-06-18T05:44:29.484Z)

Now clean up the worktree:

124. user (2026-06-18T05:44:30.774Z)

worktree removed

125. assistant (2026-06-18T05:44:45.066Z)

Done ? the diligence cleanup is up as PR [#215](https://github.com/avidullu/badminton-highlight-indexer/pull/215).

What I did

Confirmed both fixes are merged on `master` (ablation =
[#209](https://github.com/avidullu/badminton-highlight-indexer/pull/209), fusion loaders =
[#210](https://github.com/avidullu/badminton-highlight-indexer/pull/210)), which left **two**
byte-identical copies of `\_is\_fusion\_schema`. Consolidated them to a single source of truth:

- **Canonical definition** ? [`backend/eval/experiment.py`](backend/eval/experiment.py), right next to `VideoCandidates`. I picked `experiment.py` deliberately over the conceptually-tempting `fusion_golden.py`: all three consumers already import `experiment`, and `experiment` imports none of them ? so **zero** new import edges and no circular-import risk. `fusion_golden` would have dragged its heavy `TrackNetRunner`/detector deps into `ablation.py` for a pure dict-key predicate.
- The three loaders (`ablation.load_videos`, `fusion_compare.load_videos`, `fusion_audio.load_videos_with_audio`) now import the shared predicate; **bodies unchanged**, logic identical ? a true no-behaviour-change refactor.
- Removed the stale `Sequence` import from `fusion_compare` (kept in `ablation`, still used in 5 signatures); repointed `fusion_audio`'s import to the new home.
- Moved the predicate's canonical unit test into `test_experiment.py` and removed the duplicate from `test_ablation.py`.

Verification (isolated worktree off `origin/master`)

- Full suite: **855** passed, 7 skipped, coverage **80.70%** (?70 floor)
- `ruff check .` clean .`mypy backend training` clean`
- Repo-wide sweep: exactly one `def _is_fusion_schema` (in `experiment.py`), no stale imports, no `__all__` to update

Kept it to one self-contained refactor PR, branched from `origin/master`, HEAD re-verified before commit, explicit paths staged, worktree removed after push.

One small judgment call worth flagging: I kept the name underscore-prefixed (`_is_fusion_schema`) rather than promoting it to a public `is_fusion_schema`. It's an internal eval helper, and the codebase already imports underscore-prefixed helpers across modules (e.g. `_derive_video_path`), so this matches the existing convention with less churn. Easy to flip if you'd prefer it public.